OnBuch+

SIGNED COURSE • PREMIUM

COMPUTER SCIENCE • UPPER SIXTH | UPPER SIXTH — COMPUTER SCIENCE

CHAPTER 7

EVALUATING SOFTWARE DEVELOPMENT APPROACHES.

SUGGESTED TIME: 9 PERIODS

IN THIS CHAPTER

This chapter presents the complete software development life cycle: models, requirements, design, tools, quality assurance, testing, project management and maintenance. Students learn to evaluate and choose appropriate development approaches for real projects, in preparation for the GCE Advanced Level examination.

- By the end of this chapter I can describe the main activities of software development and compare life-cycle models (waterfall, iterative, incremental, spiral, agile).
- By the end of this chapter I can carry out requirements analysis and write a clear software requirements specification.
- By the end of this chapter I can explain the stages of software design and produce standard design documentation.
- By the end of this chapter I can select and justify the use of software development tools (editors, compilers, debuggers, CASE tools, version control).
- By the end of this chapter I can define software quality criteria and evaluate options for acquiring software (in-house, outsourced, off-the-shelf, open source).
- By the end of this chapter I can distinguish verification from validation and design a testing strategy (unit, integration, system, acceptance).

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE
complete, illustrated, step by step

METHODS
and worked examples

EXERCISES
exam style + solutions

SUMMARY SHEET
the essentials to remember

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+

CONTENTS

1. Software Development Activities and Life-Cycle Models
2. Requirements Analysis and Specification
3. Software Design: Stages, Notations and Documentation
4. Development Tools, Quality and Software Acquisition
5. Verification, Validation and Testing
6. Project Management, Reuse, Teamwork and Maintenance
7. Integration activity
8. Methods and worked examples
9. Exercises
10. Solutions to the exercises
11. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can describe the main activities of software development and compare life-cycle models (waterfall, iterative, incremental, spiral, agile).
- By the end of this chapter I can carry out requirements analysis and write a clear software requirements specification.
- By the end of this chapter I can explain the stages of software design and produce standard design documentation.
- By the end of this chapter I can select and justify the use of software development tools (editors, compilers, debuggers, CASE tools, version control).
- By the end of this chapter I can define software quality criteria and evaluate options for acquiring software (in-house, outsourced, off-the-shelf, open source).
- By the end of this chapter I can distinguish verification from validation and design a testing strategy (unit, integration, system, acceptance).
- By the end of this chapter I can explain software project management techniques, including planning, costing and reporting.
- By the end of this chapter I can discuss software reuse, teamwork organisation and the types of software maintenance.

PREREQUISITES

- Basic programming concepts and experience writing small programs (Lower Sixth).
- Knowledge of data types, control structures and modular programming.
- Understanding of what software is and the distinction between system and application software.
- Elementary file handling and documentation of simple programs.

1 SOFTWARE DEVELOPMENT ACTIVITIES AND LIFE-CYCLE MODELS

In 2023, a microfinance application launched in Douala crashed repeatedly during end-of-month salary payments, leaving hundreds of clients unable to withdraw their savings. Investigations revealed that the developers had skipped requirements analysis, tested only on fast Wi-Fi, and never documented their design. Yet a small team in Buea, using a disciplined development process, delivered a school-fees payment app that has run reliably for three years. What separates software that fails from software that lasts? This section answers that question by examining how professional software is planned, built, tested and maintained.

1.1 The Software Development Life Cycle (SDLC)

DEFINITION **Software Development Life Cycle (SDLC)**

The **Software Development Life Cycle (SDLC)** is a structured framework that defines the sequence of activities, deliverables and quality gates required to transform a user's need into a reliable, maintainable software product. It provides a common language for developers, managers and clients, and makes progress measurable.

KEY POINTS **Purpose of the SDLC**

- Control: breaks a complex endeavour into manageable phases with clear entry/exit criteria.
- Quality: embeds verification and validation early, reducing the cost of fixing defects (a defect found in design costs ≈ 10× less to fix than one found in production).
- Communication: gives stakeholders a shared roadmap and vocabulary.
- Traceability: links requirements → design → code → tests, enabling impact analysis when changes occur.

1.2 Core Activities of the SDLC

Every life-cycle model arranges the same fundamental activities, differing only in their ordering, iteration and emphasis.

EXPERIMENT **The Seven Core Activities**

1. Feasibility Study: Can we build it? Should we? Technical, economic, legal, opera-

tional and schedule feasibility are assessed. Output: Feasibility Report with a go/no-go recommendation.

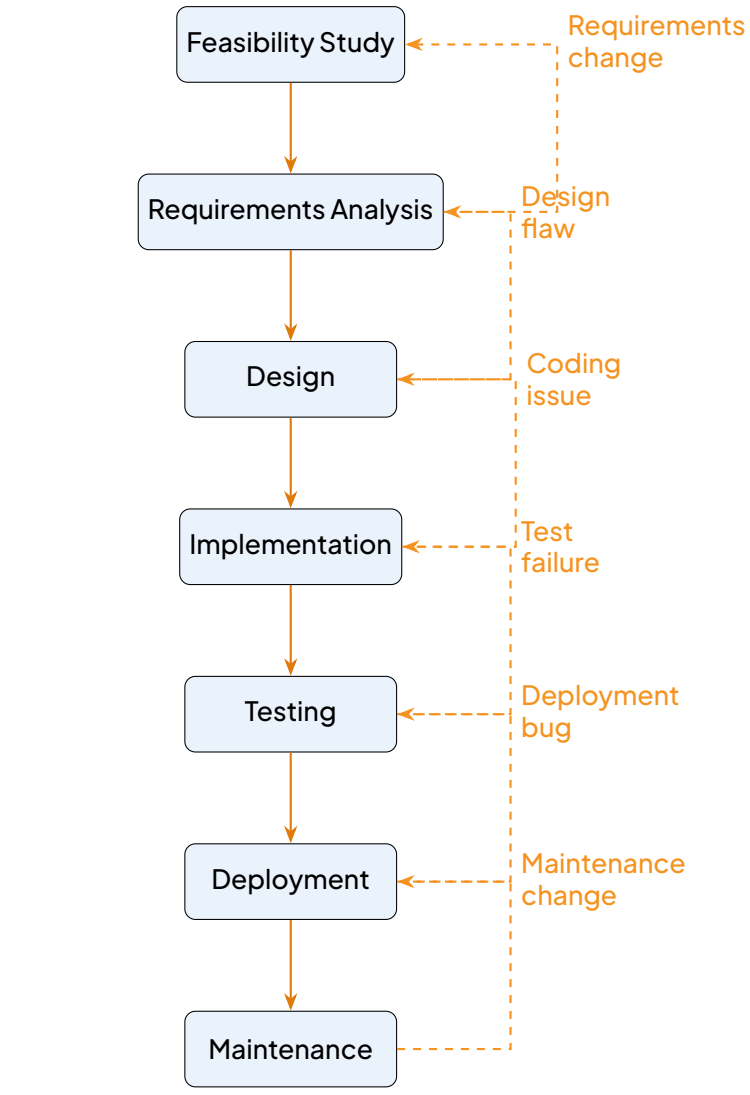
2. Requirements Analysis & Specification: What must the system do? Elicitation (interviews, workshops, observation), analysis (modelling, prioritisation), specification (SRS document), validation (reviews, prototypes).
3. Design: How will the system fulfil the requirements? Architectural design (subsystems, interfaces, data flow, technology stack) and detailed design (algorithms, data structures, database schema, UI mock-ups). Output: Software Design Description (SDD).
4. Implementation (Coding): Translating design into executable code following coding standards, version control, and peer reviews.
5. Testing: Verifying that the code meets its specification (verification) and satisfies user needs (validation). Levels: unit, integration, system, acceptance.
6. Deployment: Installation, configuration, data migration, user training, cut-over strategy (big bang, phased, parallel run).
7. Maintenance: Corrective (fix bugs), adaptive (new OS/hardware), perfective (enhancements), preventive (refactoring). Typically consumes 60–80% of total lifecycle cost.

WARNING **Common Exam Mistake**

Do not list "coding" as the first activity. The SDLC begins with understanding the problem (feasibility, requirements) before any solution is constructed.

1.3 The Waterfall Model

The **Waterfall Model** (Royce, 1970) arranges the core activities in a strict linear sequence. Each phase must be completed and signed off before the next begins; feedback loops are limited to adjacent phases.



Waterfall model with feedback arrows (feedback is possible but expensive).

PROPERTY **Waterfall Model – Advantages and Limitations**

- Advantages:
  - Simple to understand, plan and manage; clear milestones and documentation.
  - Works well when requirements are stable, well-understood and unlikely to change (e.g., embedded firmware for a known hardware platform, safety-critical systems with fixed specs).
  - Enforces discipline: design before code, test plans before testing.

- Limitations:
  - Inflexible: late changes are very expensive (require reworking all downstream phases).
  - Working software appears only late (after implementation), so stakeholder feedback is delayed.
  - Assumes requirements can be fully specified upfront – rarely true for business applications.
  - Poor risk management: high-risk technical challenges are discovered late.

1.4 Iterative and Incremental Development

Instead of one big "waterfall", the project is split into a series of smaller cycles (**iterations**), each delivering a working **increment** of the system.

DEFINITION **Prototype**

A **prototype** is a partially developed, executable version of the software built to explore requirements, validate design choices, or demonstrate feasibility. It may be throw-away (discarded after learning) or evolutionary (grows into the final product).

KEY POINTS **Iterative vs. Incremental**

- Iterative: refine the whole system through repeated cycles (e.g., build a skeleton, then flesh out all modules a little more each cycle).
- Incremental: deliver usable subsets of functionality one after another (e.g., first release: login + view balance; second release: transfer funds; third: bill payment).
- Most modern processes combine both: each iteration produces an increment.

EXEMPLE **Evolutionary Delivery – School Fees App (Buea)**

A team builds a school-fees payment system in three increments:

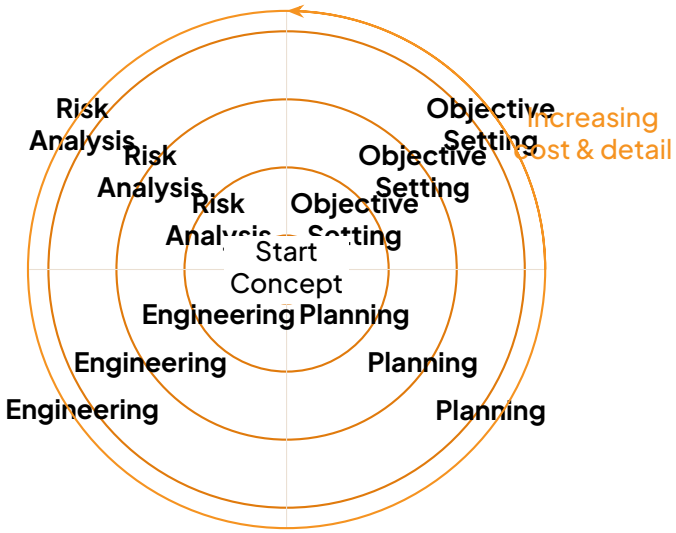
1. Increment 1 (4 weeks): Student registration, fee structure definition, cash payment recording. Deployed to pilot school.
2. Increment 2 (3 weeks): Mobile money integration (MTN MoMo, Orange Money), receipt generation, SMS notifications.

3. Increment 3 (3 weeks): Reporting dashboard for bursar, multi-school support, data export for ministry returns.

Each increment goes through mini-waterfall (require → design → code → test → deploy). Feedback from Increment 1 users reshapes Increment 2 requirements.

1.5 The Spiral Model (Boehm, 1986)

The **Spiral Model** is risk-driven. Each loop of the spiral represents a phase of the project; the radius represents accumulated cost, the angular dimension represents progress. Each loop has four quadrants:



Spiral model: four quadrants per loop – Objective Setting, Risk Analysis, Engineering, Planning.

1. **Objective Setting:** Define objectives, alternatives, constraints for this loop.
2. **Risk Analysis:** Identify and evaluate risks (technical, schedule, cost, personnel). Develop risk-mitigation strategies (prototypes, benchmarks, simulations).
3. **Engineering:** Build and test the product increment (using waterfall, prototyping, or any suitable technique) based on risk resolution.
4. **Planning:** Review the increment, plan the next loop (or decide to terminate).

PROPERTY **Spiral Model – When to Use**

- Strengths: Explicit risk management; accommodates changing requirements; suitable for large, mission-critical, innovative systems (e.g., satellite control software, banking core).

- Weaknesses: Complex to manage; requires risk-assessment expertise; overhead may be excessive for small projects.

1.6 Agile Approaches

The **Agile Manifesto** (2001) values:

Individuals and interactions	over processes and tools
Working software	over comprehensive documentation
Customer collaboration	over contract negotiation
Responding to change	over following a plan

KEY POINTS **Key Agile Principles**

- Deliver working software frequently (weeks, not months).
- Welcome changing requirements, even late in development.
- Business people and developers work together daily.
- Build projects around motivated individuals; give them trust and support.
- Face-to-face conversation is the best form of communication.
- Sustainable pace: sponsors, developers, users maintain constant pace indefinitely.
- Continuous attention to technical excellence and good design.
- Simplicity – the art of maximizing the amount of work not done – is essential.
- Self-organising teams produce the best architectures, requirements, designs.
- Regular reflection and adaptation (retrospectives).

1.7 Scrum Basics (Most Common Agile Framework)

- **Roles:** **Product Owner** (owns backlog, prioritises value), **Scrum Master** (servant-leader, removes impediments), **Development Team** (cross-functional, self-organising, 3–9 people).
- **Artefacts:** **Product Backlog** (ordered list of everything needed), **Sprint Backlog** (items selected for the current sprint + plan), **Increment** (sum of all completed backlog items, potentially shippable).
- **Events (time-boxed):**
  1. **Sprint** (2–4 weeks): fixed-length iteration.

2. **Sprint Planning:** what can be done? how will it be done?
3. **Daily Scrum (15 min):** synchronise, plan next 24h.
4. **Sprint Review:** inspect increment, adapt backlog.
5. **Sprint Retrospective:** inspect process, plan improvements.

EXEMPLE **Scrum in a Cameroonian Fintech Startup**

A Yaoundé startup builds a mobile savings app.

- Sprint 1: User registration, KYC (Know Your Customer) via ID scan, PIN setup. Demo to Product Owner (PO) and two pilot users.
- Sprint 2: Savings wallet, deposit via MoMo, interest calculation. Retrospective reveals CI pipeline too slow → invest in faster build server.
- Sprint 3: Group savings (tontine), withdrawal requests, SMS notifications. PO reprioritises: add USSD support before referral program.

Working software every two weeks; requirements evolve with market feedback.

PROPERTY **Agile vs. Plan-Driven (Waterfall/Spiral) Comparison**

Dimension	Plan-Driven (Waterfall/Spiral)	Agile (Scrum/XP/Kanban)
Requirements	Fixed, baselined early	Emergent, prioritised backlog
Planning	Detailed upfront	Just-in-time, adaptive
Documentation	Comprehensive, formal	Sufficient, living documents
Testing	Phase at the end	Continuous, automated, TDD
Customer role	Sign-off at milestones	Daily collaboration (PO)
Change control	Formal change requests	Backlog re-prioritisation
Best fit	Safety-critical, regulated, stable reqs	Innovative, uncertain, time-to-market critical

1.8 Choosing a Life-Cycle Model

No single model fits all projects. The choice depends on project characteristics.

METHOD **Selecting an Appropriate Life-Cycle Model**

1. Assess project characteristics:

- Size (person-months), team distribution, criticality (safety, financial, reputational).
- Requirements stability: well-known vs. volatile.
- Risk profile: technical novelty, integration complexity, regulatory uncertainty.
- Client availability: continuous collaboration vs. milestone reviews only.
- Schedule pressure: fixed deadline vs. flexible scope.

2. Match to model strengths:
   - Waterfall: small/medium, stable requirements, low technical risk, client unavailable daily (e.g., firmware for a known microcontroller).
   - Spiral: large, high risk, novel technology, safety-critical (e.g., air-traffic control module).
   - Iterative/Incremental: medium/large, moderate risk, requirements partially known, need early partial delivery (e.g., ERP rollout by modules).
   - Agile/Scrum: small/medium co-located team, high requirements volatility, strong client engagement, time-to-market crucial (e.g., consumer mobile app, startup MVP).
3. Consider hybrid approaches: e.g., plan-driven architecture + agile sprints for features; spiral for high-risk core, waterfall for well-understood peripherals.
4. Document the rationale in the Project Management Plan.

WORKED EXAMPLE 1 **Model Selection Justification**

Scenario: A team of 6 developers in Douala must build a web-based inventory system for a chain of 12 pharmacies. Requirements are partly known (stock, expiry, sales) but the client wants to see progress every 3 weeks and expects changes as they discover new regulatory reporting needs. The technology stack (React, Node.js, PostgreSQL) is familiar to the team. The project must launch a Minimum Viable Product in 4 months.
Solution:

1. Characteristics: Small team, co-located, familiar tech (low technical risk), volatile requirements (regulatory changes), high client availability, fixed 4-month deadline for MVP.
2. Match: Agile/Scrum fits best: short sprints (2–3 weeks) give frequent feedback; backlog accommodates changing regulations; MVP scope can be adjusted each sprint.
3. Justification for exam: “I recommend Scrum because the requirements are not fully

known and will evolve with regulatory changes, the team is small and co-located, the technology is familiar (low technical risk), and the client is available for frequent reviews. The fixed 4-month deadline for an MVP aligns with time-boxed sprints delivering incremental value."

WARNING **Exam Trap: Confusing Waterfall with Iterative Models**

- Wrong: "Waterfall is iterative because you can go back to the previous phase." Feedback loops in waterfall are exceptional, costly, and only to the immediate predecessor. True iterative models plan for rework every cycle.
- Wrong: "Agile means no documentation." Agile values working software over comprehensive documentation, not no documentation.
- Wrong: "Spiral is just waterfall with risk analysis." Spiral explicitly iterates through risk-driven loops; each loop can use any process (including prototyping) for the engineering quadrant.

DID YOU KNOW? **Did You Know?**

The first published description of the waterfall model (Royce, 1970) actually criticised the pure linear approach and recommended at least two passes (a pilot then the real thing). The "pure waterfall" taught in textbooks is a simplification that Royce himself warned against.

EXERCISE 1 **Practice Questions**

1. Define the Software Development Life Cycle (SDLC) and state its main purpose.
2. List the seven core activities of the SDLC in their logical order.
3. Draw a labelled diagram of the Waterfall model showing feedback paths.
4. Explain the four quadrants of the Spiral model.
5. A government agency requires a new tax-filing system. Requirements are fixed by law, the technology is well-understood, and the deadline is 18 months away. Which life-cycle model would you recommend? Justify your answer with three project characteristics.
6. Compare Agile and Plan-Driven approaches on: (a) requirements handling, (b) testing strategy, (c) customer involvement.

7. What is a prototype? Distinguish between throw-away and evolutionary prototypes.
8. A startup wants to launch a social-media app in 3 months. The team is 4 developers, requirements change weekly, and the founder acts as Product Owner. Which model is most suitable? Justify.
9. State two advantages and two limitations of the Waterfall model.
10. In Scrum, what is the purpose of the Sprint Retrospective?

SOLUTION **Exercise 1**

The SDLC is a structured framework defining the sequence of activities, deliverables and quality gates to transform a user need into a reliable software product. Its main purpose is to provide control, ensure quality, enable communication and maintain traceability throughout development.

SOLUTION **Exercise 2**

Feasibility Study → Requirements Analysis & Specification → Design → Implementation → Testing → Deployment → Maintenance.

SOLUTION **Exercise 3**

(See the TikZ diagram in the Waterfall section above. Ensure you label each phase and show dashed feedback arrows to the immediate predecessor.)

SOLUTION **Exercise 4**

1. Objective Setting: define objectives, alternatives, constraints. 2. Risk Analysis: identify/evaluate risks, develop mitigation (prototypes, benchmarks). 3. Engineering: build and test the increment. 4. Planning: review increment, plan next loop or terminate.

SOLUTION **Exercise 5**

Recommend Waterfall. Justification: (1) Requirements are fixed by law (stable). (2) Technology is well-understood (low technical risk). (3) Long, fixed deadline allows thorough

upfront planning and documentation required for government audit trails.

SOLUTION **Exercise 6**

(a) Plan-driven: fixed, baselined early; Agile: emergent, prioritised backlog. (b) Plan-driven: phase at end; Agile: continuous, automated, often TDD. (c) Plan-driven: milestone sign-offs; Agile: daily collaboration via Product Owner.

SOLUTION **Exercise 7**

A prototype is a partially developed executable version to explore requirements, validate design or demonstrate feasibility. Throw-away: discarded after learning. Evolutionary: refined into the final product.

SOLUTION **Exercise 8**

Scrum. Justification: small co-located team, high requirements volatility (weekly changes), strong client engagement (founder as PO), fixed short deadline (3 months) – all align with Scrum's time-boxed sprints, adaptive backlog, and frequent delivery.

SOLUTION **Exercise 9**

Advantages: simple to manage, clear milestones, works with stable requirements. Limitations: inflexible to change, working software late, poor risk management, assumes complete upfront requirements.

SOLUTION **Exercise 10**

To inspect the team's process (people, relationships, tools, Definition of Done) and create a plan for improvements to be enacted in the next Sprint.

2 REQUIREMENTS ANALYSIS AND SPECIFICATION

In the previous section we studied the software life cycle and its models (waterfall, iterative,

spiral, agile). Whatever model is chosen, every life cycle begins in the same place: before a single line of code is written, we must know what the software must do. That is the purpose of **requirements analysis and specification**. Experience in industry shows that this stage determines the success or failure of the whole project: a brilliant program that solves the wrong problem is worthless.

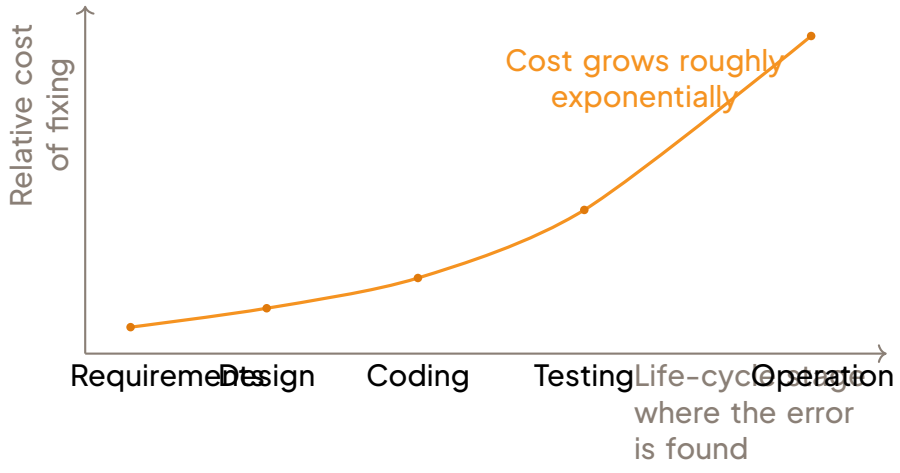
2.1 Purpose of Requirements Analysis and the Cost of Errors

Imagine a contractor in Bamenda who is asked to build "a nice house". Without a written plan — number of rooms, budget, materials — the owner and the builder will certainly disagree at the end. Software is no different, except that misunderstandings are even easier because software is invisible.

DEFINITION **Requirement**

A **requirement** is a statement describing a service that the system must provide, or a constraint on its operation or development. **Requirements analysis** is the activity of discovering, understanding, negotiating and documenting the requirements of a software system together with its stakeholders (users, customers, administrators).

The economic argument for doing this carefully is overwhelming. Studies of real projects consistently show that the later an error is discovered, the more expensive it is to fix. A requirements error detected during the requirements stage costs roughly one unit of effort; the same error discovered during design costs about 5 units; during coding about 10; during testing about 20 to 50; and after delivery it can cost 100 units or more, because design, code, tests and documentation must all be reworked.



The cost of fixing a requirements error grows dramatically the later it is discovered.

KEY POINTS **Why requirements analysis matters**

Most failed software projects fail not because of bad programming, but because of wrong, missing or misunderstood requirements. Time invested in requirements analysis is the cheapest insurance a project can buy.

2.2 Functional and Non-Functional Requirements

Requirements fall into two great families. Consider a school management system for a college in Yaoundé. "The system shall compute each student's term average" says what the system does. "The system shall respond within 2 seconds" says how well it does it.

DEFINITION **Functional requirement**

A **functional requirement** describes a specific service, behaviour or function that the system must provide: what the system shall do in response to given inputs, or what it shall not do.

DEFINITION **Non-functional requirement**

A **non-functional requirement** describes a quality attribute or a constraint on the system as a whole, rather than a specific behaviour. The main categories are:

- **Performance**: response time, throughput, memory usage (e.g. "generate a report card in under 3 s").
- **Usability**: ease of learning and use (e.g. "a new secretary shall be able to register a student after 2 hours of training").
- **Reliability**: availability and failure tolerance (e.g. "the system shall be available 99% of school hours").
- **Security**: protection of data and access control (e.g. "only the principal shall modify marks after validation").
- **Portability**: ability to run in different environments (e.g. "the system shall run on Windows and Linux").

WARNING **Vague requirements are not acceptable**

Statements such as "the system shall be fast", "the system shall be user-friendly" or "the system shall be reliable" are not acceptable requirements, because nobody can test

them. Every requirement must be **measurable**: replace "fast" by "respond within 2 s for 95% of requests under a load of 50 simultaneous users".

2.3 Requirements Elicitation Techniques

Requirements are not invented by the analyst; they are elicited (drawn out) from stakeholders. No single technique is sufficient, so analysts combine several:

- **Interviews**: structured or open discussions with stakeholders. Excellent for deep understanding and for discovering political or organisational issues; but time-consuming and dependent on the interviewer's skill.
- **Questionnaires**: written questions distributed to many users. Cheap and good for collecting statistics from a large population (e.g. all teachers of a school); but questions cannot be adapted on the fly and answers are often shallow.
- **Observation**: watching users actually perform their current work (e.g. observing how the bursar records fees). Reveals what people really do, which often differs from what they say they do; but the observer may disturb normal work.
- **Document analysis**: studying existing forms, reports, regulations and the manuals of the current system (e.g. report card templates, MINESEC regulations). Gives precise, factual domain knowledge; but documents may be outdated.
- **Use cases and user stories**: modelling typical interactions between actors and the system to capture functional requirements in a structured, testable way (e.g. "As a teacher, I want to enter marks for my class so that report cards can be generated").

2.4 User Requirements and System Requirements

Requirements are written at two levels of detail, for two different audiences.

DEFINITION **User vs system requirements**

User requirements are high-level statements, in natural language, of the services the system must provide and the constraints under which it must operate; they are written for customers and managers who are not technical. **System requirements** are detailed, precise and structured descriptions of the system's functions, inputs, outputs and constraints; they form the basis of the contract between customer and developer and the starting point of design.

EXEMPLE **Two levels of the same requirement**

User requirement: "The system shall allow the principal to print the term report cards of a

whole class."
System requirement: "Given a class identifier and a term number, the system shall produce, for every enrolled student, a PDF report card containing the marks of all subjects, the weighted average computed with the official coefficients, the class rank, and the principal's signature field; generation of a class of 60 students shall take less than 30 s."

2.5 The Software Requirements Specification (SRS) Document

The results of requirements engineering are recorded in a contract-like document.

DEFINITION **Software Requirements Specification**

The **Software Requirements Specification (SRS)** is the official document that records all functional and non-functional requirements of the system to be developed. It serves as a contract between customer and developer, as the reference for design and testing, and as the baseline against which changes are managed.

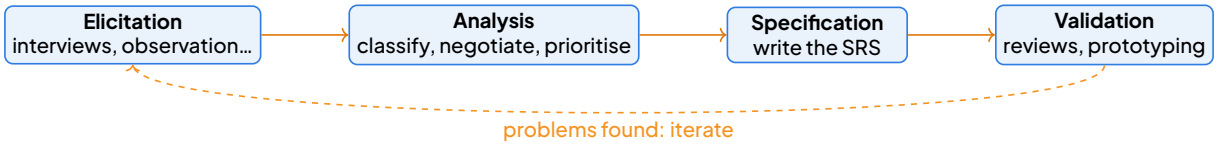
A typical SRS contains: an **introduction** (purpose, scope, definitions, references); an **overall description** (product perspective, user classes, assumptions, general constraints); the **functional requirements** (numbered, one per service); the **non-functional requirements** (performance, usability, reliability, security, portability); **external interface requirements** (user, hardware, software interfaces); and appendices (glossary, data dictionaries).

ID	Requirement statement	Type	Source
FR-01	The system shall compute each student's term average using the official subject coefficients.	Functional	Principal
FR-02	The system shall allow a teacher to enter marks only for the classes he or she teaches.	Functional	Teachers
NFR-01	A report card shall be generated in less than 3 s.	Performance	Secretary
NFR-02	All passwords shall be stored encrypted; marks shall be modifiable only before validation.	Security	Principal
NFR-03	The system shall run on Windows 10 and Ubuntu Linux.	Portability	IT staff

Extract of an SRS requirements table for a school management system.

2.6 The Requirements Engineering Process

Requirements engineering is not a single action but a cycle of four activities, repeated until the specification is accepted.



The requirements engineering process: a spiral of elicitation, analysis, specification and validation.

2.7 Requirements Validation

Before design begins, the SRS must be validated: checked that it truly describes the system the customer needs. Four quality criteria are examinable:

- **Completeness**: every service and constraint needed by the stakeholders is present; nothing is missing.
- **Consistency**: no two requirements contradict each other (e.g. "marks are modifiable at any time" contradicts NFR-02 above).
- **Testability**: each requirement is stated precisely enough that one can design a test proving whether the delivered system satisfies it.
- **Traceability**: each requirement is uniquely identified and linked to its source and, later, to the design elements and tests that implement and check it, so that the impact of any change can be found.

METHOD **Writing a testable requirement**

1. Start with "The system shall…" and use one numbered sentence per requirement.
2. State a single, observable behaviour or a measurable quality — never two requirements merged with "and".
3. Attach a measurable criterion: a number, a limit, a list of cases (time in seconds, percentage, screen resolution…).
4. Give it a unique identifier (FR-01, NFR-03) and record its source for traceability.
5. Ask: "Could a tester write a pass/fail test from this sentence alone?" If not, rewrite it.

2.8 Common Problems in Practice

Three difficulties plague almost every real project. **Ambiguity**: natural language is imprecise

— "the system shall handle large classes" means 40 students to one reader and 200 to another; the cure is measurable wording and formal notations where needed. **Incomplete requirements**: stakeholders forget rare cases (a student repeating a class, a teacher leaving mid-term); the cure is systematic observation, document analysis and checklists. **Changing requirements**: the school changes its grading policy while the software is being built; the cure is to freeze a baseline SRS, manage every change through a formal change-control procedure, and prefer iterative life-cycle models when change is expected.

DID YOU KNOW? **Requirements in the Cameroonian context**

In many Cameroonian schools and businesses, requirements are often expressed orally or in informal notes. A disciplined SRS, even a lightweight one, prevents costly misunderstandings — for instance, when a developer assumes "term average" means a simple mean while the official bulletin requires a weighted average with specific coefficients. Writing it down saves weeks of rework.

WORKED EXAMPLE 2 **Classifying and improving requirements**

The following statements were collected for a library system in a lycée in Douala. Classify each as functional (F) or non-functional (NF), and rewrite the unacceptable ones so that they become testable.
(a) "The system shall record the loan of a book to a student." (b) "The system shall be fast." (c) "The system shall compute the fine for a book returned late, at 100 FCFA per day." (d) "The system shall be easy to use."

SOLUTION

Solution.
(a) F — it describes a service (recording a loan). Already testable: create a loan, check it is stored.
(b) NF (performance) but not acceptable as written. Testable version: "The system shall display the result of a book search within 2 s for a catalogue of 10 000 titles."
(c) F — a precise computation rule; testable with a worked example (return 3 days late ⇒ fine of 300 FCFA).
(d) NF (usability) but not acceptable as written. Testable version: "A librarian shall be able to register a new book after at most 1 hour of training, with no more than 2 errors."
Exam tip: ask "does it say what the system does (F) or how well / under which constraint (NF)?" — then check that a number makes it testable.

EXERCISE 2 **From interview to SRS**

During an interview, the principal of a college says: "Parents must be able to consult their children's results on their phones, and the system must never lose data, and it must work

even when many parents connect at the end of term."
1. Identify two functional and two non-functional requirements hidden in this statement.
2. Rewrite each requirement so that it is measurable and testable.
3. Give each requirement an identifier and state which validation criterion (completeness, consistency, testability, traceability) your rewriting improves.

SOLUTION **Exercise 1**

1. Functional: (F1) parents can consult results online; (F2) results are stored persistently (no data loss). Non-functional: (NF1) availability/reliability — data must not be lost; (NF2) performance — the system must support the end-of-term peak load. (F2 and NF1 are two views of the same need; accepting either is correct if justified.)
2. F1: "The system shall allow a parent, after login, to view the term results of his or her child on a screen of at least 5 inches." NF1: "No committed transaction shall be lost; daily backups shall be kept for 30 days." NF2: "The system shall serve 200 simultaneous parents with a response time under 3 s."
3. Identifiers (F1, NF1, NF2) improve traceability; measurable criteria improve testability; separating the needs improves completeness and helps detect contradictions (consistency).

KEY POINTS **Key points**

Requirements analysis discovers what the system must do; errors made here are the most expensive of all. Distinguish **functional** (services) from **non-functional** requirements (performance, usability, reliability, security, portability); elicit them by interviews, questionnaires, observation, document analysis and use cases; express them as user requirements then system requirements; record them in an **SRS**; and validate the SRS for completeness, consistency, testability and traceability. Every requirement must be numbered, sourced and measurable.

3 SOFTWARE DESIGN: STAGES, NOTATIONS AND DOCUMENTATION

In the previous section we learnt how to capture and specify what a system must do: the requirements document is the contract between the client and the development team. But a contract is not a building. Between the requirements and the final program lies a crucial intellectual activity: **software design**. Design answers the question how — how the system will be structured, how its parts will communicate, and how data will be organised — **before** a single line of code is written.

Think of constructing a school campus in Bamenda. The proprietor's needs ("we need classrooms for 500 students, a laboratory, a refectory") are the requirements. The architect's plans — site plan, floor plans, electrical and plumbing diagrams — are the design. Only then do the masons begin to lay blocks — the implementation. Nobody would dream of building a school without plans; yet many failed software projects are exactly "buildings without plans".

3.1 The Purpose of Software Design

DEFINITION **Software design**

Software design is the creative activity that transforms a requirements specification into a detailed description of how the software will be built: its architecture, its components and their interfaces, and its data organisation. Its output is a set of **design documents** (models, diagrams and descriptions) that guide the programmers.

Design is a bridge: on one side, the requirements say what the system must do; on the other side, the code says exactly what the machine executes. The designer's job is to make decisions — about structure, data, interfaces and algorithms — so that coding becomes almost mechanical.

WARNING **Design is not coding**

A very common mistake at GCE level (and in real projects!) is to confuse design with implementation. Design decides what each part does and how the parts fit together; implementation is the act of writing the code in a programming language. Designing a module that "validates a student's mark between 0 and 20" is design; typing the `if` statement in Python is implementation. Good design happens before coding, on paper or in diagrams.

3.2 Architectural Design

Architectural design is the first and highest-level design stage. It identifies the main subsystems of the software and how they are organised and communicate.

DEFINITION **Software architecture**

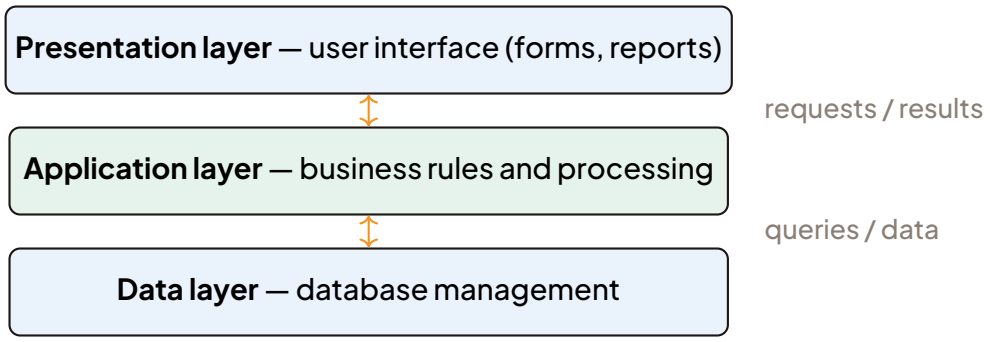
The **software architecture** of a system is the high-level organisation of the system into components (subsystems), the relationships and interactions between those components, and the principles guiding their design and evolution. It is the "skeleton" of the system.

Two architectural styles are explicitly required by the syllabus.

Layered architecture. The system is organised into horizontal layers, where each layer pro-

vides services to the layer above and uses services of the layer below. A typical information system (for example, the school fees management system of a college in Yaoundé) has three layers:

- **Presentation layer**: the user interface — forms, menus, reports.
- **Application (logic) layer**: the business rules — computing averages, checking that fees are paid before printing a report card.
- **Data layer**: storage and retrieval — the database of students, marks and payments.



Layered architecture of a typical information system: each layer only talks to its immediate neighbour.

The great advantage is separation of concerns: you can change the user interface (e.g. move from a desktop application to a web page) without rewriting the business rules or the database.

Client-server architecture. The system is split between **clients**, which request services, and a **server**, which provides them. In a cybercafé management system, the cashier's computer (client) sends requests to a central machine (server) holding the database of sessions and payments. Many clients can share one server; the server centralises the data, which guarantees consistency. The weakness is that the server is a single point of failure: if it breaks down, every client stops.

Architecture	Main advantage	Main weakness
Layered	Layers are independent: one layer can be modified or replaced without touching the others	Requests must cross several layers, which can slow processing
Client-server	Data centralised on the server: consistency, shared access, easier backup	The server is a single point of failure and can be overloaded

3.3 Interface Design

Once the architecture is fixed, we must design how the parts communicate. There are two kinds of interfaces.

User interface (UI) design concerns the interaction between the software and the human user: screens, menus, forms, error messages, reports. Good UI design follows principles such as consistency (similar actions look similar), feedback (the system always tells the user what is happening), and error tolerance (invalid input is rejected politely, e.g. "Mark must be between 0 and 20"). For a school system used by a bursar who may not be a computer expert, a clear, forgiving interface is not a luxury — it is a requirement.

Component interface design concerns how software modules call each other: which data a module receives (inputs), which results it returns (outputs), and the exact format of that data. A component interface is like the menu of a restaurant: it tells you what you can order, not how the kitchen prepares it. For example, a module `computeAverage` might have the interface: input — a list of marks and their coefficients; output — a real number, the weighted average. Any module respecting this interface can replace another without disturbing the rest of the system.

3.4 Component and Data Design

Component design refines the architecture by decomposing each subsystem into **modules** — smaller, manageable units (procedures, functions, classes). This modular decomposition applies the ancient strategy divide and conquer: a problem too large to solve at once is split into subproblems that can be designed, coded and tested separately.

Data design selects the **data structures** (arrays, records, files, lists) used inside the program and designs the **database** when persistent storage is needed. Database design typically identifies the **entities** (e.g. Student, Book, Loan), their **attributes** (name, class, date of birth) and the **relationships** between them (a student borrows a book). Poor data design is a classic source of failure: if marks are stored as text instead of numbers, computing an average becomes unnecessarily painful.

3.5 Design Principles: Modularity, Cohesion, Coupling, Information Hiding

Why decompose into modules at all? Because a single monolithic program of 50 000 lines is impossible to understand, test or repair. Four principles guide good decomposition.

- **Modularity**: the system is divided into independent modules, each with a well-defined task and interface.
- **Cohesion**: the degree to which the elements inside a module belong together. A module that only handles student marks is highly cohesive; a module that handles marks, prints reports and manages network connections is not.
- **Coupling**: the degree of dependence between modules. If changing one module forces changes in many others, coupling is high.
- **Information hiding**: each module hides its internal details (data structures, algorithms) and exposes only its interface. Other modules cannot — and need not — know what is inside.

PROPERTY **High cohesion, low coupling**

A good modular design aims at high cohesion within modules and low coupling between modules. High cohesion makes a module easy to understand and reuse; low coupling

means a change or an error in one module does not propagate through the whole system. This property should be stated and justified in the examination.

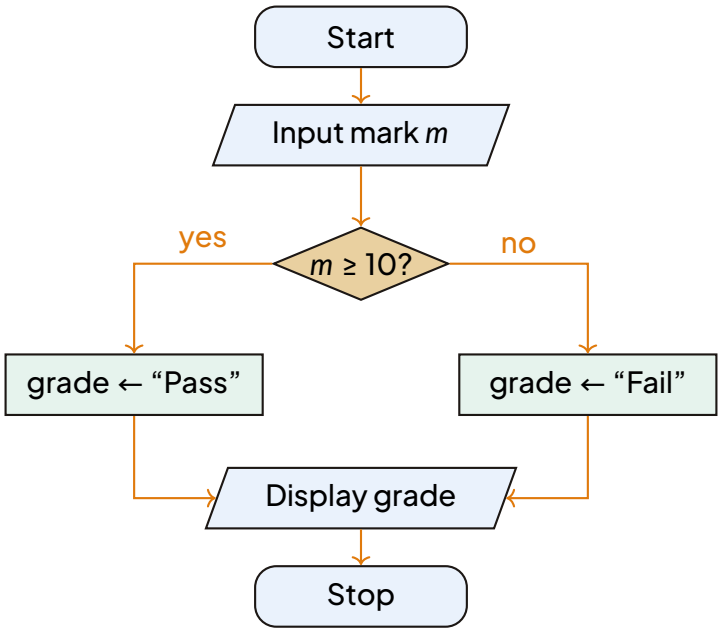
EXEMPLE **Cohesion and coupling in a school system**

Good design: module `MarksManager` computes averages and ranks (high cohesion); module `ReportPrinter` only formats and prints. They communicate through a single, simple interface: `ReportPrinter` receives a ready-made list of results (low coupling). If the grading formula changes, only `MarksManager` is modified.
Bad design: one giant module `DoEverything` reads marks, computes, prints and writes files, with global variables shared everywhere. Changing the printing format risks breaking the average calculation — high coupling, low cohesion.

3.6 Design Notations

Designers communicate through **diagrams**, not long prose. The syllabus requires four notations.

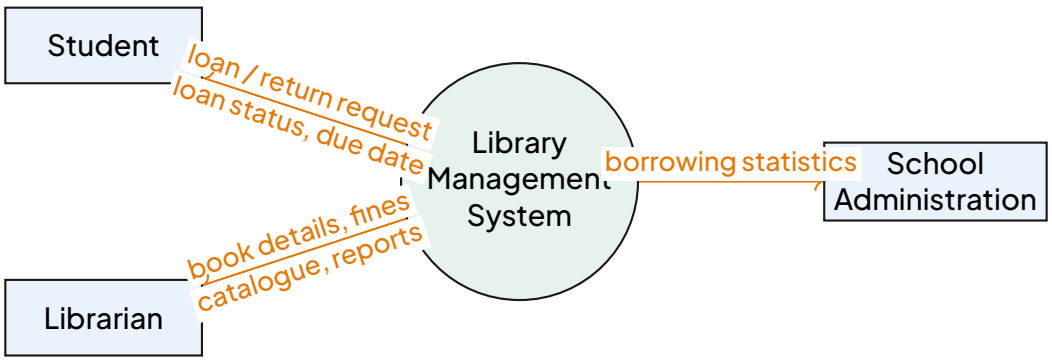
Flowcharts describe the logic of a single module or algorithm: sequence, selection and iteration, using standard symbols (oval for start/stop, parallelogram for input/output, rectangle for processing, diamond for decision).



Flowchart of a simple module that decides whether a mark (out of 20) is a pass or a fail.

Data flow diagrams (DFDs) show how data moves through the system, using four symbols: **ex-**

ternal entities (sources or sinks of data, drawn as rectangles), **processes** (transformations, drawn as circles or rounded boxes), **data stores** (files or databases, drawn as open rectangles) and **data flows** (labelled arrows). A level-0 (context) diagram shows the **whole system as one process** with its external entities.

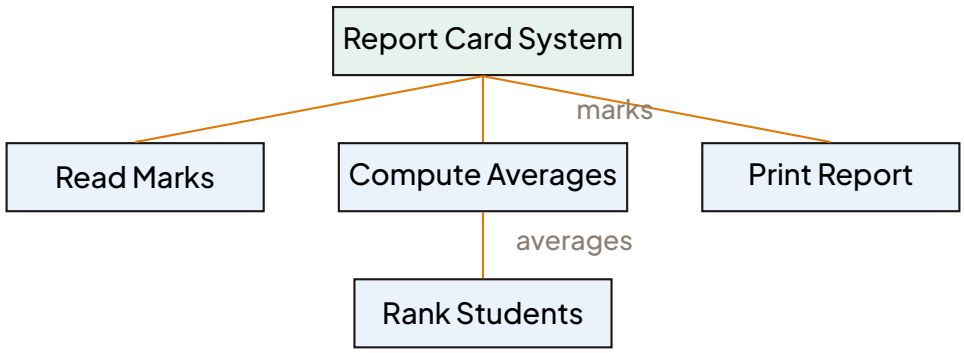


Level-0 (context) data flow diagram for a library management system: the whole system is one process exchanging data with external entities.

METHOD **Drawing a level-0 data flow diagram**

1. Identify the external entities: the people, organisations or other systems that send data to or receive data from your system (e.g. Student, Librarian).
2. Draw one single process in the centre, labelled with the name of the whole system, numbered 0.
3. List the data flows: for each entity, determine what data it sends in and what data it receives out; draw a labelled arrow for each.
4. Check the balance: every arrow must connect an entity to the process; there are no data stores and no direct entity-to-entity flows at level 0.
5. Label clearly: every flow must carry a meaningful data name (noun phrase), never a verb.

Structure charts show the hierarchical decomposition of a system into modules: a tree where the top module calls sub-modules, annotated with the data passed between them. They answer "which module calls which?".



Structure chart of a report card system: the top module calls sub-modules; labels show data passed between them.

UML (Unified Modeling Language) is the standard modern family of design notations. Two diagrams matter at this level:

- **Use case diagram**: shows the actors (users or external systems, drawn as stick figures) and the use cases (services the system offers, drawn as ellipses). For the library: actor Student connected to use cases Borrow book and Return book. It summarises the system from the users' point of view.
- **Class diagram**: shows the classes of the system (e.g. Book, Member, Loan), their attributes and operations, and the relationships between them (e.g. one Member may have many Loans). It is the blueprint for object-oriented implementation.

Notation	What it shows	Typical question it answers
Flowchart	Logic of one module: sequence, selection, iteration	In what order are the steps executed?
DFD	Movement of data between entities, processes and stores	Where does each piece of data come from and go to?
Structure chart	Hierarchy of modules and data passed between them	Which module calls which?
UML use case	Actors and the services the system offers them	What can each type of user do?
UML class diagram	Classes, attributes, operations and relationships	What objects exist and how are they linked?

3.7 Design Documentation

All the decisions above must be recorded in a **design document**. A typical design document contains:

- an overview of the architecture (with the main architectural diagram);
- a description of each module: its purpose, interface (inputs/outputs) and internal logic (flowchart or pseudocode);

- the data design: data structures, file layouts, database schema;
- the user interface designs (screen sketches, report formats);
- references to the requirements each design element satisfies (traceability).

Why is this documentation so important? First, **maintenance**: most of a system's life is spent after delivery, correcting and adapting it. A maintenance programmer who was not in the original team can only work safely if the design is documented. Second, **teamwork**: in a team, the designer of module A and the coder of module B must agree on the interface between them — the document is that agreement. Third, **communication with the client**: diagrams such as use cases can be validated with non-technical users before coding begins.

WORKED EXAMPLE 3 **Interpreting a level-0 DFD**

Statement. A canteen management system for a lycée in Douala is shown as a single process. External entities: Student and CanteenManager. The student sends a meal order and receives a ticket number; the manager sends daily menu and prices and receives a sales summary. (a) Draw the level-0 DFD. (b) State why no data store appears.

SOLUTION

Solution. (a) Draw one central circle labelled 0. Canteen Management System. Rectangle Student on the left: arrow meal order from Student to the process; arrow ticket number from the process to Student. Rectangle CanteenManager on the right: arrow daily menu and prices from Manager to the process; arrow sales summary from the process to Manager.
(b) A level-0 (context) diagram shows only the system as a whole and its exchanges with the outside world. Data stores (such as the file of daily sales) are internal to the system; they appear only when the system is decomposed in the level-1 DFD.

WARNING **Exam tip: interpreting and completing diagrams**

In the GCE examination you may be given a partially completed DFD or flowchart and asked to complete or interpret it. Remember: (1) in a DFD, label flows with data names, never with verbs; data cannot flow directly between two external entities or between two data stores — it must pass through a process; (2) in a flowchart, every decision diamond has exactly two labelled exits (yes/no), and every path must eventually reach Stop; (3) check that every process has at least one input flow and one output flow. Practise by redrawing the library DFD above from memory, then checking the flows.

EXERCISE 3 **Designing a module**

A module must read the three term averages of a student (each out of 20) and display

"Admitted" if the general average is at least 10, otherwise "Repeats".

1. Draw the flowchart of this module.
2. State the interface of the module (inputs and outputs).
3. Explain, using cohesion and coupling, why the calculation and the screen display should be in two separate modules.

SOLUTION **Exercise 1**

1. Flowchart: Start → input t_1, t_2, t_3 → $avg \leftarrow (t_1 + t_2 + t_3)/3$ → decision $avg \geq 10$? → yes: display "Admitted"; no: display "Repeats" → Stop.
2. Inputs: three real numbers t_1, t_2, t_3 in $[0, 20]$. Output: a message ("Admitted" or "Repeats"), or equivalently the computed average and the decision.
3. A module that only computes the average and the decision has high cohesion (one clear task) and can be reused, for instance to produce a class ranking without any screen display. The display module has its own single task. Keeping them separate gives low coupling: changing the screen format (colours, language) never risks breaking the calculation, and the calculation can be tested independently.

KEY POINTS **Key points**

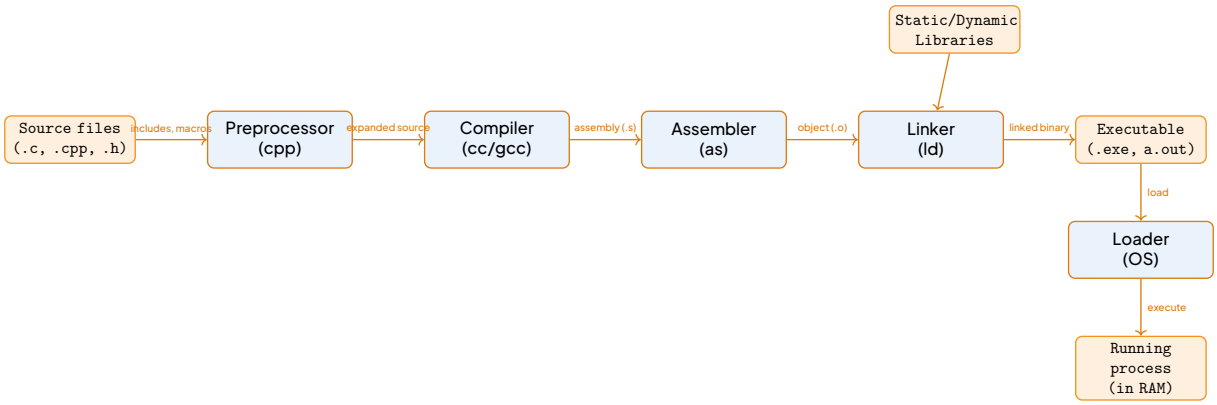
- Design bridges what (requirements) and how (implementation); it happens before coding.
- Architectural design fixes the overall structure: layered and client-server architectures.
- Interface design covers user interfaces and component interfaces; component and data design decompose the system into modules and choose data structures and databases.
- Aim for high cohesion and low coupling; apply information hiding.
- Master the notations: flowcharts (module logic), DFDs (data movement), structure charts (module hierarchy), UML (use case and class diagrams).
- The design document records all decisions; it is essential for maintenance, teamwork and client validation.

4 DEVELOPMENT TOOLS, QUALITY AND SOFTWARE ACQUISITION

In the previous section we saw how a software system is designed: from architectural decisions down to detailed algorithms and data structures, documented with UML diagrams, data dictionaries and interface specifications. A design, however precise, remains a paper artefact until it is transformed into executable code. This section explores the toolchain that turns source code into a running product, the quality attributes we must verify, and the strategic acquisition choices (build, buy, reuse) that confront every project team. Understanding these topics is essential for Paper 2 questions on project management, tool selection and justification of software procurement decisions (Lessons 31, 32, 34, 35).

4.1 Development Tools

The Compile–Link–Execute Toolchain When a programmer writes source code in a high-level language (C, C++, Java, C#), several distinct tools cooperate to produce an executable program. Figure summarises the classic compile–link–execute pipeline for a compiled language such as C or C++.



The classic compile–link–execute toolchain for a compiled language (C/C++).

DEFINITION **Editor, Compiler, Interpreter, Linker, Debugger**

- Editor: A program for writing and modifying source code (e.g. VS Code, Vim, Notepad++). Modern editors provide syntax highlighting, auto-completion and integrated terminal.
- Compiler: Translates the entire source program into machine code (or intermediate bytecode) before execution. Errors are reported all at once. Examples: `gcc`, `javac`, `clang`.

- Interpreter: Reads, translates and executes source statements one by one at runtime. No separate executable is produced. Examples: `python3`, `node`, `php`.
- Linker: Combines separately compiled object files and library modules into a single executable, resolving external symbols (function calls, global variables).
- Debugger: Allows controlled execution: breakpoints, step-by-step execution, inspection of variables and call stack (e.g. `gdb`, VS Code debugger, `lldb`).

WARNING **Compiler vs Interpreter — Common Confusion**

- A language is not inherently compiled or interpreted; it is the implementation that chooses. Python is usually interpreted (CPython), but PyPy uses a JIT compiler. Java is compiled to bytecode, then interpreted/JIT-compiled by the JVM.
- In exams, if asked "State one advantage of a compiler over an interpreter", answer: Compiled programs generally run faster because translation is done once; interpreters translate repeatedly at runtime.
- Conversely, interpreters give faster edit-test cycles and better portability (same source runs on any platform with the interpreter).

Integrated Development Environments (IDEs) and CASE Tools Individual tools (editor, compiler, debugger) can be used separately, but professional development almost always uses an Integrated Development Environment.

DEFINITION **Integrated Development Environment (IDE)**

An **IDE** is a software application that provides comprehensive facilities for software development in a single graphical interface: source editor, build automation (compiler/linker), debugger, version-control integration, code navigation, refactoring tools, and often a visual designer for GUIs or databases. Examples: Visual Studio, IntelliJ IDEA, Eclipse, NetBeans, PyCharm, Code::Blocks.

DEFINITION **CASE Tool**

Computer-Aided Software Engineering (CASE) tools automate activities across the software life cycle. Upper CASE tools support analysis and design (requirements modelling, UML diagramming, code generation from models). Lower CASE tools support implementation, testing and maintenance (compilers, debuggers, test generators, version

control). Integrated CASE (I-CASE) tools cover the whole life cycle and share a central repository (data dictionary).

KEY POINTS **IDE vs CASE Tool**

- An IDE is primarily a programmer's workbench for coding, building and debugging.
- A CASE tool is a systems engineering tool for modelling, documentation, traceability and sometimes code generation.
- Modern IDEs incorporate many lower-CASE features (refactoring, unit-test runners, static analysis). Upper-CASE modelling (e.g. Enterprise Architect, Visual Paradigm) often integrates with IDEs via plugins.

Version Control and Configuration Management No serious project survives without version control (also called source control or revision control).

DEFINITION **Version Control System (VCS)**

A VCS records changes to files over time so that specific versions can be recalled later. It supports:

- Commit: Snapshot of the project at a point in time with a descriptive message.
- Branch: Independent line of development (e.g. `feature/login`, `bugfix/123`).
- Merge: Integrating changes from one branch into another.
- Tag: Permanent label for a release (e.g. `v1.0.0`).
- Remote repository: Central server (GitHub, GitLab, Bitbucket, self-hosted) for collaboration.

METHOD **Basic Git Workflow (Feature Branch Model)**

1. `git clone <repo-url>` — obtain a local copy.
2. `git checkout -b feature/xyz` — create and switch to a new branch.
3. Edit files, then `git add <files>` — stage changes.
4. `git commit -m "Concise message"` — record snapshot locally.

5. `git push origin feature/xyz` — publish branch to remote.
6. Open a Pull Request / Merge Request for code review.
7. After approval, merge into `main` (or `develop`) and delete the feature branch.
8. `git pull` — update local `main` with latest changes.

WARNING **Git Traps for Students**

- Forgetting to pull before pushing: Causes rejected non-fast-forward updates. Always `git pull` before pushing (use `git pull --rebase` only if you understand rebasing).
- Committing large binaries: Bloats repository history. Use `.gitignore` and Git LFS for assets.
- Vague commit messages: "Fixed bug" is useless. Write: "Fix off-by-one in tax calculation (issue #42)".
- Confusing `merge` vs `rebase`: Merge preserves history; rebase rewrites it. Never rebase shared branches.

4.2 Software Quality

What Is Software Quality? Quality is not merely "absence of bugs". It is the degree to which a system satisfies stated and implied needs of its stakeholders under specified conditions.

DEFINITION **Software Quality (ISO/IEC 25010 Perspective)**

Software quality is the capability of a software product to satisfy stated and implied needs when used under specified conditions. It is characterised by a quality model comprising eight top-level characteristics (ISO/IEC 25010:2011):

1. Functional Suitability: Functional completeness, correctness, appropriateness.
2. Reliability: Maturity, availability, fault tolerance, recoverability.
3. Performance Efficiency: Time behaviour, resource utilisation, capacity.
4. Compatibility: Co-existence, interoperability.
5. Usability: Appropriateness recognisability, learnability, operability, user error protection, accessibility.

6. Security: Confidentiality, integrity, non-repudiation, accountability, authenticity.
7. Maintainability: Modularity, reusability, analysability, modifiability, testability.
8. Portability: Adaptability, installability, replaceability.

KEY POINTS **Key Quality Attributes — Exam Definitions**

- Correctness: The extent to which software meets its specification.
- Reliability: Probability of failure-free operation for a specified time in a specified environment.
- Usability: Ease with which users can learn, operate and recover from errors.
- Efficiency: Ratio of useful work to resources consumed (CPU, memory, bandwidth).
- Maintainability: Ease of modifying software to correct faults, improve performance or adapt to change.
- Portability: Ease of transferring software to another hardware/OS/platform.

EXEMPLE **Quality Trade-offs in a Mobile Banking App (Cameroon Context)**

A fintech startup in Douala builds a mobile money app. Stakeholders prioritise:

- Security (confidentiality, integrity) — non-negotiable for financial data.
- Usability — low-literacy users, small screens, intermittent 3G/4G.
- Reliability — transactions must not be lost during network drops.
- Efficiency — minimal data usage, fast on low-end Android devices.

Trade-off: Adding strong encryption (security) increases CPU cycles (efficiency). The team chooses AES-256 with hardware acceleration, accepts slightly larger APK size (portability), and invests in offline-first architecture (reliability). Each decision is documented in a Quality Attribute Workshop (QAW) report.

4.3 Software Acquisition Options

Organisations rarely build everything from scratch. The build vs buy vs reuse decision is strategic.

DEFINITION **Acquisition Strategies**

- In-house development (Build): Organisation's own team designs, codes, tests and maintains the software. Full control, full cost, full IP ownership.
- Outsourcing (Contract development): External vendor builds to specification. Fixed-price or time-and-materials. Requires rigorous contract, acceptance criteria, IP clauses.
- Commercial Off-The-Shelf (COTS): Ready-made product purchased/licensed (e.g. Microsoft Dynamics, SAP, QuickBooks). Fast deployment, licence fees, limited customisation.
- Open Source Software (OSS): Software with source code available under a licence granting freedom to use, study, modify, distribute (e.g. Linux, PostgreSQL, Odoo, ERPNext). No licence fee, but not zero cost.

METHOD **Evaluating an Acquisition Option — Structured Approach**

1. Define requirements: Functional, non-functional, regulatory, integration, scalability.
2. Identify candidates: Market scan, RFI (Request for Information), shortlist 3–5 options.
3. Score against criteria: Use a weighted scoring matrix (see Table).
4. Total Cost of Ownership (TCO): Licence/subscription + hardware + implementation + customisation + training + support + maintenance + exit costs over 3–5 years.
5. Risk assessment: Vendor lock-in, roadmap alignment, community health (for OSS), data sovereignty, compliance (e.g. Cameroon Personal Data Protection Law).
6. Proof of Concept (PoC): Prototype critical workflows with top 2 candidates.
7. Decision and negotiation: Present business case to steering committee; negotiate SLA, escrow, source-code access.

Criterion	In-house Build	Outsourcing	COTS	Open Source
Initial cost	High (staff, infra)	High (contract)	Medium (licence)	Low (no licence)
Time to market	Slow (months–years)	Medium (contract)	Fast (weeks)	Medium (config)
Customisation	Full	High (per contract)	Low–Medium	High (source access)
Control & IP	Full ownership	Contract-defined	Vendor owned	Community governed
Ongoing cost	Salaries, infra	SLA fees	Subscription, upgrades	Support, hosting
Support	Internal team	Vendor SLA	Vendor SLA	Community or paid vendor
Scalability	Design for it	Negotiate in contract	Limited by product	Design for it
Risk	Team turnover	Vendor dependency	Vendor lock-in	Community abandonment

Decision table comparing software acquisition options. Adapt weights to project context.

WARNING **Open Source ≠ Zero Cost — The Hidden Costs**

- Implementation & customisation: Developers' time to adapt, integrate, test.
- Hosting & infrastructure: Cloud VMs, Kubernetes, backups, monitoring.
- Support: No vendor SLA unless you pay a third party (e.g. Red Hat for Linux, EnterpriseDB for PostgreSQL).
- Security patching: Your team must track CVEs, apply updates, test regressions.
- Technical debt: Forking upstream makes future upgrades painful.
- Compliance: Verify licence compatibility (GPL vs MIT vs Apache) — legal review needed.

Exam tip: If a scenario says "The school wants a free library management system", do not write "Download open source, cost = 0". Write: "Evaluate OSS (e.g. Koha, Evergreen) against TCO: hosting, customisation for Cameroon curriculum, staff training, annual support contract."

4.4 Software Evaluation Criteria and Checklists

When comparing specific products (e.g. three COTS ERP systems), a structured checklist ensures objectivity.

DEFINITION **Software Evaluation Checklist Categories**

1. Functional Fit: Mandatory vs desirable features; workflow match; configurability vs customisation.
2. Technical Architecture: Technology stack, API openness, microservices vs monolith, cloud-native, database standards.
3. Quality Attributes: Performance benchmarks, security certifications (ISO 27001, SOC 2), accessibility (WCAG 2.1 AA).
4. Vendor Viability: Financial health, market share, roadmap transparency, reference customers in similar domain/size.
5. Licensing & Legal: Perpetual vs subscription, per-user vs per-core, audit rights, data ownership, exit clauses, escrow.
6. Support & SLA: Response times, severity levels, support channels, language/time-zone coverage, upgrade policy.
7. Total Cost of Ownership (5 years): Licence, hardware/cloud, implementation, training, customisation, annual maintenance, exit/migration.
8. User Experience: UI consistency, mobile/responsive, role-based dashboards, offline capability.
9. Integration: Pre-built connectors (REST, SOAP, message queues), support for Cameroon payment gateways (MTN MoMo, Orange Money), national ID verification APIs.
10. Regulatory Compliance: Data localisation, OHADA accounting standards, Cameroon Personal Data Protection Law.

EXEMPLE **Weighted Scoring Example — Choosing a School Management System**

A bilingual secondary school in Yaoundé evaluates three systems. Weights reflect priorities (sum = 100).

Criterion (Weight)	Weight	System A (COTS)	System B (OSS)	System C (In-house)
Functional fit (25)	25	22	18	24
TCO 5 years (20)	20	12	18	10
Support/SLA (15)	15	14	10	12
Customisation (15)	15	8	14	15
Integration (MoMo, GCE) (10)	10	9	7	10
Vendor viability (10)	10	9	8	9
Compliance (5)	5	5	4	5
Total	100	79	79	85

Interpretation: System C (in-house) scores highest but assumes the school has a capable dev team. If not, System A (COTS) is safer despite higher TCO. System B (OSS) needs a support contract. The decision must weigh scores and organisational capacity.

WORKED EXAMPLE 4 **Justifying an Acquisition Choice**

Scenario: A Cameroonian NGO needs a case-management system for 50 social workers across 5 regions. Requirements: offline-first (poor connectivity), multilingual (French/English), compliance with Cameroon Personal Data Protection Law, budget 15 million FCFA over 3 years. Options: (1) Custom build by local dev agency (25M FCFA), (2) COTS SaaS (12M FCFA/yr), (3) OSS (CiviCRM) hosted on Azure (setup 5M, hosting 3M/yr, support 2M/yr).
Solution:

1. Eliminate Option 1: Exceeds budget (25M > 15M).
2. Option 2 (COTS SaaS): 36M over 3 years > budget. Also, SaaS rarely offers true offline-first; data residency may violate Cameroon law.
3. Option 3 (OSS CiviCRM): Setup 5M + 3yrs × (3M+2M) = 5M + 15M = 20M. Still over budget.
4. Revised Option 3: Host on local server (existing hardware), use community support, train one internal admin. Cost: 5M setup + 3yrs × 1M (admin time) = 8M. Fits budget. Offline-first via local network sync. Source code access allows French/English UI. Data stays in Cameroon.
5. Recommendation: Adopt CiviCRM on-premise with internal admin. Allocate 2M for contingency. Document risk: community support only; mitigate by training two admins.

Examiner's comment: The candidate identifies constraints, calculates TCO, eliminates

infeasible options, proposes a viable variant, and acknowledges residual risk. Full marks.

EXERCISE 4 **Practice Questions**

1. A startup must choose between building a custom CRM or subscribing to Salesforce. List four factors they should evaluate, and for each state whether it favours build or buy.
2. Explain the difference between a compiler and an interpreter. Give one scenario where an interpreter is preferable.
3. A school downloads an open-source library system (GPL v3). They modify the source code and deploy it on their intranet only (no external access). Do they have to release their modifications? Justify your answer.
4. Draw a diagram showing the stages from source code to execution for a Java program (include `javac`, `.class`, JVM, JIT).
5. Define maintainability and portability as software quality attributes. Give one design decision that improves each.

SOLUTION **Exercise 1**

1. Factors: (a) Time-to-market — favours buy (SaaS ready in weeks). (b) Unique business processes — favours build (custom fit). (c) Long-term TCO — favours build if high volume, buy if low volume. (d) Data sovereignty/regulatory — may favour build (on-premise control) or buy (vendor with local data centres).
2. Compiler translates whole program before execution; interpreter translates and executes statement by statement. Interpreter preferable for rapid prototyping or teaching where immediate feedback matters.
3. No. GPL v3 requires source distribution only when the modified program is conveyed (distributed) to others. Internal deployment on an intranet is not conveyance. (AGPL would trigger obligation for network-accessible services.)
4. Diagram: `.java` $\xrightarrow{\text{javac}}$ `.class (bytecode)` $\xrightarrow{\text{Class Loader}}$ JVM $\xrightarrow{\text{Interpreter/JIT}}$ Machine code $\xrightarrow{\text{CPU}}$ Execution.
5. Maintainability: Ease of modification. Improved by modular design, low coupling, high cohesion. Portability: Ease of moving to another platform. Improved by platform-independent languages (Java, Python), abstraction layers (PDO for DB),

containerisation (Docker).

DID YOU KNOW? **Cameroon Tech Ecosystem — Local Hosting & Open Source**

The Agence de Régulation des Télécommunications (ART) and the Ministry of Posts and Telecommunications (MINPOSTEL) encourage local data hosting. Several Cameroonian universities (e.g. University of Yaoundé I, University of Buea) run Moodle (OSS) on local servers for e-learning, avoiding foreign cloud costs and latency. The Government Interoperability Framework (CIF) promotes open standards and open-source components for public-sector systems. When evaluating acquisition, always check whether a "local hosting" requirement exists — it often rules out pure SaaS.

5 VERIFICATION, VALIDATION AND TESTING

In the previous section, we explored the tools that support development and the criteria for acquiring quality software. We now turn to the critical phase that determines whether the software we have built actually works as intended and satisfies the user's needs. Writing code is only half the battle; the other half is providing evidence that the code is correct. As the famous computer scientist Edsger Dijkstra warned, "Program testing can be used to show the presence of bugs, but never to show their absence!" This section equips you with the rigorous terminology, techniques, and strategies used in the industry and examined by the GCE Board to ensure software reliability.

5.1 Verification versus Validation: The Fundamental Distinction

Students often confuse these two terms. In the GCE examination, distinguishing them precisely earns easy marks. The distinction was famously captured by Barry Boehm:

DEFINITION **Verification**

Verification is the process of evaluating a system or component to determine whether the products of a given development phase satisfy the conditions imposed at the start of that phase. In short: "Are we building the product right?" It checks conformance to the specification (design documents, code standards, requirements document).

DEFINITION **Validation**

Validation is the process of evaluating a system or component during or at the end of the

development process to determine whether it satisfies specified requirements. In short: "Are we building the right product?" It checks conformance to the user's actual needs and the operational environment.

KEY POINTS **The "Two Rights" Rule**

- Verification: Matches product → Specification (Internal consistency, standards, logic).
- Validation: Matches product → User Needs (External fitness for purpose, usability, business value).

EXEMPLE **Verification vs Validation in a Cameroonian Context**

Imagine a team develops a "Mobile Money Fee Calculator" for a telecom operator in Douala.

- Verification: The code correctly implements the formula `fee = amount * 0.02 + 50` as written in the Technical Specification Document. The variable names follow the company coding standard. The loop terminates.
- Validation: The user (the telco) tests it and realises the government just changed the tax law: fees are now capped at 10,000 FCFA. The software works exactly as specified (verified), but it is useless for the current law (failed validation).

5.2 Static Verification: Finding Faults Without Execution

Static verification analyses the artefacts (requirements, design diagrams, source code) without executing the program. It is the first line of defence and is generally cheaper than fixing bugs found later.

PROPERTY **Static Verification Techniques**

1. Reviews / Inspections: A formal, planned meeting where authors present a work product (code, design) to a team of reviewers who examine it against checklists (coding standards, logic errors, security vulnerabilities). Roles: Moderator, Author, Reviewer, Scribe.
2. Walkthroughs: Less formal. The author guides the team through the artefact, explaining the logic. The goal is learning and early defect detection.

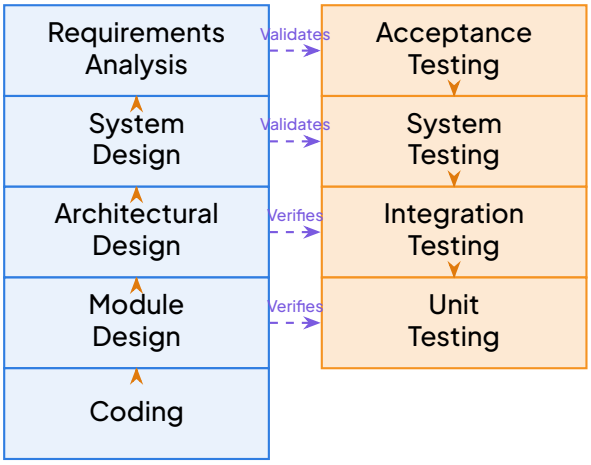
3. Static Analysis Tools: Automated tools (linters, SonarQube, Coverity) that scan source code for syntax violations, potential null pointers, unreachable code, and complexity metrics (cyclomatic complexity).

WARNING **Common Exam Trap**

Do not confuse a Walkthrough (author-led, informal, educational) with an Inspection (moderator-led, formal, checklist-driven, defect-focused). Inspections are more rigorous and find more defects per hour.

5.3 Dynamic Testing: The V-Model and Testing Levels

Dynamic testing involves executing the code with test data. The industry standard for visualising the relationship between development phases and testing phases is the **V-Model**.



The V-Model: Linking Development Stages (Left) to Testing Levels (Right). Verification dominates the lower left; Validation dominates the upper right.

DEFINITION **Unit Testing**

Testing individual modules, functions, or classes in isolation. Usually performed by the developer. Uses stubs (simulate called modules) and drivers (call the module under test).

DEFINITION **Integration Testing**

Testing the interfaces and interaction between integrated units/modules.

- Big Bang: All modules combined at once (hard to debug).
- Top-Down: Test high-level modules first using stubs for lower levels.
- Bottom-Up: Test low-level modules first using drivers.
- Sandwich (Hybrid): Combines both.

DEFINITION **System Testing**

Testing the complete, integrated system against the System Requirements Specification (SRS). Covers functional and non-functional requirements (performance, security, stress, recovery). Performed by an independent QA team.

DEFINITION **Acceptance Testing**

Formal testing conducted by the client/user to determine whether to accept the system.

- Alpha Testing: Performed at the developer's site by end-users (or simulated users) in a controlled environment.
- Beta Testing: Performed at the user's site by real users in a live/uncontrolled environment. Feedback goes to developers for final fixes.

KEY POINTS **Exam Tip: Alpha vs Beta vs Acceptance**

- Alpha: Dev site, controlled, developers watch.
- Beta: User site, real world, no developers present.
- Acceptance: The decision gate. Can be Alpha, Beta, or a separate formal contract test (Factory Acceptance Test / Site Acceptance Test).

5.4 Testing Techniques: Black-Box vs White-Box

How do we choose test inputs? Two fundamental approaches exist.

DEFINITION **Black-Box Testing (Functional / Specification-Based)**

The internal code structure is unknown/ignored. Test cases are derived solely from the specification/requirements. Techniques:

1. Equivalence Partitioning: Divide input domain into classes where the system should behave identically. Test one value from each class.
2. Boundary Value Analysis (BVA): Errors cluster at boundaries. Test values at, just below, and just above the edges of partitions.
3. Decision Table Testing: For complex business logic with multiple conditions.
4. State Transition Testing: For systems with defined states (e.g., ATM, Traffic Light).

DEFINITION **White-Box Testing (Structural / Glass-Box)**

The internal code structure is known. Test cases are derived from the code logic (paths, branches, statements). Coverage Metrics (Exam Favourites):

- Statement Coverage: Every executable statement executed at least once.
- Branch/Decision Coverage: Every branch (true/false) of every decision point executed.
- Path Coverage: Every possible path through the code executed (often infeasible for loops).
- Condition Coverage: Each boolean sub-expression evaluates to true and false.

METHOD **Designing Black-Box Test Cases (Equivalence Partitioning & BVA)**

Scenario: A function `grade(mark)` returns a letter grade for the GCE A-Level (0–100 scale).

1. Identify Valid Partitions:
   - P_1: [0, 39] → Fail (F)
   - P_2: [40, 49] → Pass (E)
   - P_3: [50, 59] → Pass (D)
   - P_4: [60, 69] → Pass (C)
   - P_5: [70, 100] → Pass (A/B)
2. Identify Invalid Partitions:

- P_6: (−∞, −1] (Negative)
- P_7: [101, +∞) (Too high)
- P_8: Non-integer / String inputs.

3. Select Boundary Values (BVA): For each partition boundary b, test $b - 1$, b, $b + 1$.
   - Boundary 0: Test −1, 0, 1
   - Boundary 39/40: Test 39, 40, 41
   - Boundary 49/50: Test 49, 50, 51
   - Boundary 59/60: Test 59, 60, 61
   - Boundary 69/70: Test 69, 70, 71
   - Boundary 100: Test 99, 100, 101
4. Document Test Cases: Create a table (Test ID, Input, Expected Output, Partition Tested).

WORKED EXAMPLE 5 **Worked Example: Test Case Table for** `grade(mark)`

Statement: Design a black-box test suite for the grading function described above using Equivalence Partitioning and Boundary Value Analysis. Present as a table.
Solution:

Test ID	Input Mark	Expected Grade	Technique / Partition	Type
TC01	-1	Error / Invalid	Invalid Partition (Negative)	Negative
TC02	0	F	Valid Partition P_1, Boundary (Min)	Valid / Boundary
TC03	1	F	Valid Partition P_1	Valid
TC04	39	F	Valid Partition P_1, Boundary (Max)	Valid / Boundary
TC05	40	E	Valid Partition P_2, Boundary (Min)	Valid / Boundary
TC06	41	E	Valid Partition P_2	Valid
TC07	49	E	Valid Partition P_2, Boundary (Max)	Valid / Boundary
TC08	50	D	Valid Partition P_3, Boundary (Min)	Valid / Boundary
TC09	51	D	Valid Partition P_3	Valid
TC10	59	D	Valid Partition P_3, Boundary (Max)	Valid / Boundary
TC11	60	C	Valid Partition P_4, Boundary (Min)	Valid / Boundary
TC12	61	C	Valid Partition P_4	Valid
TC13	69	C	Valid Partition P_4, Boundary (Max)	Valid / Boundary
TC14	70	B	Valid Partition P_5, Boundary (Min)	Valid / Boundary
TC15	85	A	Valid Partition P_5 (Mid-range)	Valid
TC16	100	A	Valid Partition P_5, Boundary (Max)	Valid / Boundary
TC17	101	Error / Invalid	Invalid Partition (Overflow)	Negative
TC18	“ABC”	Error / Invalid	Invalid Partition (Type)	Negative

Commentary: This table demonstrates traceability from requirements (partitions) to test cases. Note the focus on boundaries (39, 40, 41) where off-by-one errors typically hide.

5.5 Test Planning, Coverage and the Limits of Testing

A **Test Plan** (IEEE 829 standard) is the master document. It defines scope, approach, resources,

schedule, and risks.

DEFINITION **Test Plan Structure (Key Sections)**

1. Test Plan Identifier: Unique ID.
2. Introduction/Objectives: What is being tested? (Module X, System Y).
3. Test Items (Scope): Features to be tested / NOT tested.
4. Approach/Strategy: Levels (Unit, Int, Sys), Techniques (Black/White box), Tools (JUnit, Selenium).
5. Pass/Fail Criteria: e.g., "Zero Critical bugs, < 5 Minor bugs, 90% Statement Coverage".
6. Suspension/Resumption Criteria: When to stop testing (e.g., blocker bug found).
7. Test Deliverables: Test cases, scripts, logs, summary report.
8. Risks & Contingencies: "Key tester on leave", "Test environment unstable".
9. Schedule/Staffing: Who does what, when.

WARNING **The Fundamental Limit of Testing – Dijkstra's Principle**

Testing shows the presence of defects, not their absence. Even 100% code coverage does not guarantee correctness.

- Missing Logic: Code covers a path, but the path implements the wrong formula (specification error).
- Exhaustive Testing is Impossible: For a function with two 32-bit integers inputs, there are 2^{64} combinations. At 1 billion tests/sec, it takes 584 years.
- The Pesticide Paradox: Running the same tests repeatedly eventually stops finding new bugs. Tests must be regularly reviewed and updated.

KEY POINTS **Test Coverage Criteria – What GCE Expects**

- Statement Coverage: Weakest. 'if (x>0) y=1;' covered by x=1. Misses x=-1.
- Branch Coverage: Stronger. Requires x=1 (True) and x=-1 (False).
- Path Coverage: Strongest but impractical for loops (infinite paths).

- Exam Rule: 100% Branch Coverage $\Longrightarrow$ 100% Statement Coverage. Reverse is FALSE.

5.6 Debugging and Regression Testing

Once a test fails, the process shifts from testing (finding failure) to debugging (locating and fixing the defect).

DEFINITION **The Debugging Cycle**

1. Reproduction: Create a minimal, repeatable test case that triggers the failure.
2. Diagnosis / Localization: Hypothesize causes. Use: Print statements, Debuggers (breakpoints, watch variables, call stack), Logging, Binary Search (comment out half the code), Rubber Ducking (explaining code aloud).
3. Fix: Apply the correction. Warning: Fixes often introduce new bugs.
4. Verification (Regression Testing): Re-run the failed test case AND a subset of the existing test suite to ensure the fix didn't break existing functionality.

DEFINITION **Regression Testing**

Selective re-testing of a system after modifications to ensure that:

1. The reported defect is fixed (Confirmation Testing).
2. No new defects have been introduced in unchanged areas (Regression Testing proper).
3. No new defects introduced in the fixed area.

Strategies: Re-test All (expensive, safe), Selective (risk-based, select tests covering changed modules), Prioritized (run critical/high-priority tests first). Automation is essential for frequent regression (CI/CD pipelines).

DID YOU KNOW? **Cameroon Context: Mobile Banking Apps**

In Cameroon, apps like Mobile Money (MTN/Orange), Express Union Mobile, or UBA Leo undergo rigorous regression testing before every update. A bug in a "Transfer" module after a "UI Colour Change" update would be a catastrophic regression failure. Automated test suites (thousands of scripts) run overnight on device farms to catch these before release.

5.7 Integration Activity: The V-Model Traceability Matrix

As an integration activity for this chapter, you are expected to demonstrate traceability from Requirements → Design → Code → Test Cases. This is the essence of the V-Model.

EXPERIMENT **Guided Activity: Building a Traceability Matrix**

Scenario: Requirement `REQ-001`: "The system shall calculate VAT at 19.25% for all taxable goods in Cameroon." Task: Fill the traceability matrix below.

Req ID	Design Element (Module/Class)	Code Unit (Function)	Test Case IDs
REQ-001	`TaxCalculationModule` (Sequence Diagram)	`calculateVAT(amount)` in `TaxService.java`	TC-VAT-01 (Valid), TC-VAT-02 (Zero), TC-VAT-03 (Negative), TC-VAT-04 (Boundary)

Instruction: In your practical project (Lesson 36), you must produce a similar matrix for your major features. Examiners check: Does every requirement have at least one test case? Does every test case map to a requirement? Orphan test cases or untested requirements are quality failures.

EXERCISE 5 **GCE Style Practice Questions**

1. Distinguish between Verification and Validation. Give one example of an activity for each in the context of developing a School Management System. (4 marks)
2. Explain the difference between Alpha Testing and Beta Testing. State where each takes place and who performs it. (4 marks)
3. A function `calculateDiscount(age, isStudent)` returns a discount percentage.
   - Age < 5: 100% (Free)
   - Age 5–18: 50% if Student, else 0%
   - Age 19–60: 10% if Student, else 0%
   - Age > 60: 30% (Senior)

   Design a black-box test case table using Equivalence Partitioning and Boundary Value Analysis. Include Test ID, Inputs (age, isStudent), Expected Output, and Technique used. (10 marks)
4. Define Regression Testing. Why is automation critical for effective regression testing in Agile environments? (3 marks)

5. True/False with Justification: “Achieving 100% Statement Coverage guarantees the program is bug-free.” (3 marks)

SOLUTION **Exercise 1**

Verification: “Building the product right.” Checking artefacts against specifications. Example: Code review of the `calculateFees()` function against the design pseudocode; checking naming conventions. Validation: “Building the right product.” Checking final product against user needs. Example: User Acceptance Testing (UAT) where the school bursar tests the fee receipt printing on the actual thermal printer used in the office.

SOLUTION **Exercise 2**

Aspect	Alpha Testing	Beta Testing
Location	Developer’s site (Lab/Controlled)	User’s site (Live/Real environment)
Performed By	End-users (invited) / QA team simulating users	Actual end-users (customers)
Control	High (developers observe, debuggers attached)	Low (no developers present, real data)
Goal	Catch major crashes, usability issues early	Real-world compatibility, performance, feedback

SOLUTION **Exercise 3 – Outline**

Partitions (Age): Invalid (-ve), [0,4], [5,18], [19,60], [61, MaxInt]. Partitions (isStudent): True, False. Combinatorial Explosion: Test cross-product of boundaries. Key Boundaries (Age): -1, 0, 4, 5, 18, 19, 60, 61, MaxInt. Sample Rows: TC01: Age=-1, Stud=T → Error (Invalid) TC02: Age=0, Stud=F → 100% (Boundary Min Valid) TC03: Age=4, Stud=T → 100% TC04: Age=5, Stud=T → 50% (Boundary 5) TC05: Age=5, Stud=F → 0% TC06: Age=18, Stud=T → 50% (Boundary 18) TC07: Age=19, Stud=T → 10% (Boundary 19) TC08: Age=60, Stud=F → 0% (Boundary 60) TC09: Age=61, Stud=F → 30% (Boundary 61) TC10: Age=100, Stud=T → 30%

SOLUTION **Exercise 4**

Regression Testing: Re-running functional and non-functional tests after code changes (fixes, enhancements, config changes) to ensure previously working software still works. Why Automation? Agile has short sprints (2 weeks). Manual re-testing of the full suite every sprint is impossible. Automation allows "Continuous Testing" in the CI pipeline (run on every commit), giving instant feedback.

SOLUTION **Exercise 5**

FALSE. Justification: Statement coverage only ensures every line is executed. It does not guarantee: 1. Correct logic (e.g., 'x = a + b' executed, but spec required 'a - b'). 2. Boundary conditions handled correctly (loop runs once, but fails at 0 or max iterations). 3. Missing paths (code not written for a requirement). 4. Data corruption / integration errors. Dijkstra: Testing shows presence, not absence of bugs.

6 PROJECT MANAGEMENT, REUSE, TEAMWORK AND MAINTENANCE

In the previous section we saw how verification, validation and testing give us confidence that a piece of software does what it should. But testing does not happen by magic: somebody must decide who tests, when, with what resources, and what happens after the software is delivered. Experience shows that real projects fail far more often because of poor management than because of poor programming. In this final section we study how software projects are organised and monitored, how teams work together, how existing software can be reused, and why **maintenance** — what happens after delivery — consumes most of the money spent on software over its lifetime.

6.1 Software Project Management

Aims of project management.

Imagine a team in Douala asked to build a school-fees management system for a group of colleges. Without management, programmers start coding immediately, nobody knows the deadline, two people write the same module, and the budget is exhausted before testing begins. **Software project management** is the discipline that prevents this chaos.

DEFINITION **Software project management**

Software project management is the set of activities concerned with planning, organising, monitoring and controlling a software project so that the product is delivered on

time, within budget and meeting the agreed requirements.

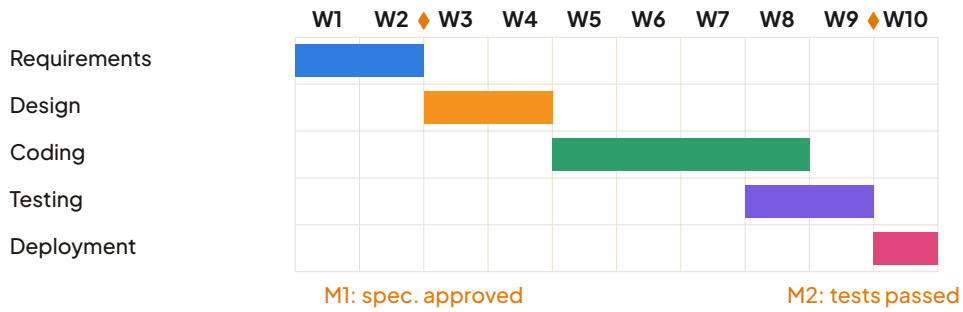
The manager's first task is **planning**: breaking the project into **tasks** (activities), estimating the **duration** and **resources** of each task, identifying **dependencies** (task B cannot start before task A finishes), and assigning tasks to team members. The result of planning is a **schedule**. During execution the manager **monitors** actual progress against the schedule and **controls** the project by taking corrective action (reassigning staff, renegotiating deadlines) whenever reality drifts away from the plan.

Scheduling with Gantt charts.

The most common way to present a schedule is the **Gantt chart**: a bar chart in which each horizontal bar represents one task, drawn against a time axis. At a glance, everyone can see what should be happening on any given date, which tasks run in parallel, and where the checkpoints are.

METHOD **Building a simple Gantt chart**

1. List the tasks of the project (e.g. from the life-cycle phases: analysis, design, coding, testing, deployment).
2. Estimate the duration of each task in days or weeks.
3. Identify dependencies: which tasks must finish before others can start? Some tasks can run in parallel.
4. Draw a time axis (columns = weeks) and one row per task.
5. Draw a bar for each task, starting at the earliest week allowed by its dependencies, with length equal to its duration.
6. Mark milestones (diamond symbols) at key checkpoints, e.g. "requirements approved".



Gantt chart of a small project over 10 weeks. Testing overlaps coding (weeks 8–9): modules are tested as soon as they are ready.

DEFINITION **Milestone and deliverable**

A **milestone** is a significant, checkable point in the project (e.g. "design document approved"); it has zero duration. A **deliverable** is a tangible product handed over to the client or to the next phase (e.g. the requirements specification, the source code, the user manual). Milestones are usually confirmed by the production of a deliverable.

WARNING **Milestone ≠ task**

Candidates often draw milestones as bars on a Gantt chart. A milestone is an instant, not an activity: it consumes no time and no resources. Represent it with a diamond, never with a bar of non-zero length.

6.2 Cost Estimation and Risk Management

Cost estimation.

Before signing a contract, a company must estimate the project **cost**, dominated by staff effort measured in **person-months** (one person-month = the work of one person during one month). Common approaches are:

- **Expert judgement**: experienced managers estimate from similar past projects.
- **Estimation by analogy**: compare with a completed project of similar size and adjust for differences.
- **Algorithmic models**: formulas such as COCOMO, which estimate effort from the expected size of the code (in thousands of lines of code).
- **Bottom-up estimation**: estimate each task separately, then add the estimates.

All estimates are uncertain; a wise manager adds a safety margin (contingency) rather than promis-

ing the most optimistic figure.

DEFINITION **Risk management**

A **risk** is an undesirable event that may occur and would harm the project. **Risk management** is the cycle of: (1) identification — listing possible risks; (2) analysis — estimating each risk's probability and impact; (3) monitoring — watching for warning signs during the project; (4) mitigation — planning actions that reduce the probability or the impact.

Typical software risks include: a key programmer resigning, requirements changing mid-project, underestimated task durations, and failure of a third-party component. Typical mitigations include: documenting knowledge so that no person is irreplaceable, prototyping unclear requirements early, scheduling slack time, and testing third-party components before depending on them.

6.3 Project Reporting and Documentation Standards

A project that is not reported on is a project out of control. The manager produces regular **progress reports** (weekly or monthly) stating: work completed since the last report, work planned next, milestones reached or missed, current expenditure versus budget, and problems encountered with proposed solutions. Reports keep the **client**, **senior management** and the **team** informed and allow early correction of delays: a delay discovered at week 3 can still be repaired; the same delay discovered at week 11 cannot.

All project documents (specifications, design documents, test plans, reports, minutes of meetings) must follow **documentation standards**: agreed rules on document structure, naming, version numbering, dating and approval signatures. Standards guarantee that any team member — or any future maintainer — can find and understand any document, and that everyone works on the current version rather than an obsolete copy.

6.4 Team Organisation and Communication

Software is built by teams, not lone geniuses. A typical team includes:

Role	Main responsibilities
Project manager	Plans, schedules, assigns tasks, manages risks, reports progress.
Systems analyst	Studies the problem, gathers and specifies requirements with the client.
Designer	Produces the architectural and detailed design (data structures, interfaces).
Programmer	Implements the design in code; performs unit testing.
Tester	Designs and runs system tests; reports and tracks defects.

In small Cameroonian companies one person may play several roles, but the responsibilities remain distinct and must all be covered. Good **communication** is essential: regular team meetings, shared document repositories, coding standards and clear written specifications prevent the classic disaster where two programmers write incompatible modules. A common organisational style is the **chief-programmer team**: one senior engineer leads and takes the key

technical decisions, supported by programmers, a librarian (responsible for documents and versions) and testers.

6.5 Software Reuse

DEFINITION **Software reuse**

Software reuse is the practice of building new systems partly or wholly from existing software artefacts rather than writing everything from scratch.

Reuse exists at several **levels**:

- **Code-level reuse**: copying functions or using libraries (e.g. a maths library, a date-handling routine).
- **Component-level reuse**: integrating self-contained units with well-defined interfaces (e.g. a chart-drawing component, a login module, a database connector).
- **System-level reuse**: configuring and combining whole existing applications or frameworks (e.g. adapting an open-source school-management package instead of writing one).

Benefits: faster development, lower cost, increased reliability (reused code is already tested in real use), and standardisation across products. **Difficulties**: finding and understanding suitable components, components that do not quite fit the requirements, licence and cost issues, and the "not invented here" attitude of some programmers. Reuse also creates a dependency: if the reused component has a bug, your system inherits it, and if the supplier disappears, you inherit the maintenance burden.

DID YOU KNOW? **Reuse in practice**

Much of the world's software runs on reused open-source components: web servers, encryption libraries and database systems are almost never rewritten from scratch. A Cameroonian start-up building a mobile-money application will reuse existing payment and SMS components and concentrate its effort on what is genuinely new — its own business rules and user experience.

6.6 Software Maintenance

Delivery is not the end of the story — it is the beginning of the longest phase of the software life cycle.

DEFINITION **Software maintenance and its four types**

Software maintenance is all the work done on a software system after it has been delivered to keep it useful. The four recognised types are:

- Corrective: fixing faults (bugs) discovered in operation.
- Adaptive: modifying the software to work in a changed environment (new operating system, new hardware, new tax law).
- Perfective: improving the software — new features, better performance, better interface — at users' request.
- Preventive: restructuring, re-documenting or updating the software to make future maintenance easier and avoid future faults.

Why does maintenance dominate cost? Because a successful system lives for many years — often 10 to 20 — while development takes months. During those years bugs surface, laws change, users demand improvements, and platforms evolve. Studies of real systems consistently attribute roughly **two-thirds or more** of the total lifetime cost of software to maintenance, with perfective and adaptive work far outweighing simple bug-fixing.



Typical distribution of total lifetime software cost: maintenance absorbs the largest share.

WARNING **Maintenance is not just fixing bugs**

A very common exam error is to classify every maintenance request as corrective. Ask yourself: what triggered the change? A crash on wrong input → corrective. A new version of the operating system or a new VAT rate → adaptive. Users asking for an extra report → perfective. Rewriting tangled code so it is easier to modify later → preventive. The trigger, not the coding activity, determines the type.

6.7 Legacy Systems and Documentation for Maintainability

DEFINITION **Legacy system**

A **legacy system** is an old system, still in active use and often critical to an organisation, built with outdated technologies, poorly documented, and difficult and expensive to maintain or replace.

Many banks and administrations still depend on legacy systems written decades ago. Replacing them is risky (they embody years of business rules, some of which nobody remembers explicitly), so they must be maintained as they are. The key weapon against maintenance cost is **documentation for maintainability**: up-to-date requirement and design documents, commented and well-structured source code, a user manual, a maintenance log recording every change (what, why, who, when), and version control. When the original programmers have left, documentation is the only memory of why the system was built the way it was.

WORKED EXAMPLE 6 **Planning and maintaining a school results system**

A team of 5 people must deliver an exam-results management system for a college in Bamenda in 12 weeks. (a) Propose a schedule with milestones. (b) Identify two risks and their mitigations. (c) Six months after delivery, the college asks for a new ranking report and the ministry changes the grading scale. Classify each request.

SOLUTION

(a) Schedule. Requirements (weeks 1–2, milestone M1: specification approved), design (weeks 3–4, M2: design document delivered), coding (weeks 5–9), testing (weeks 8–11, overlapping coding, M3: acceptance tests passed), deployment and training (week 12). Deliverables: specification, design document, tested code, user manual.
(b) Risks. Risk 1: the only database programmer falls ill or resigns — mitigation: pair work and up-to-date technical documentation so another member can take over. Risk 2: requirements change when the college sees a prototype — mitigation: show a prototype early (week 4) and freeze requirements after M1, with later changes handled as paid maintenance.
(c) Classification. The new ranking report is an improvement requested by users → perfective maintenance. The changed grading scale is imposed by a change in the external environment → adaptive maintenance. Neither is corrective, since nothing was "broken".

EXERCISE 6 **Risks, reuse and maintenance types**

A company in Yaoundé is building a mobile application for paying water bills, reusing an existing payment component. (1) List three project risks and one mitigation for each. (2) State two benefits and two difficulties of reusing the payment component. (3) Classify: (i) fixing a crash when the network drops; (ii) adapting the app to a new Android ver-

sion; (iii) adding a consumption-history graph; (iv) restructuring the code to ease future changes.

SOLUTION **Exercise 1**

(1) Possible answers: underestimation of coding time (mitigate with bottom-up estimation plus slack); loss of a key developer (mitigate with documentation and knowledge sharing); failure of the reused payment component (mitigate with an early integration test and an alternative supplier). (2) Benefits: faster, cheaper development; the component is already tested in real use. Difficulties: it may not exactly match the requirements; licence costs and dependency on the supplier. (3) (i) corrective; (ii) adaptive; (iii) perfective; (iv) preventive.

KEY POINTS **Key points**

- Project management = planning, scheduling (Gantt charts), monitoring and controlling, so that software is delivered on time, within budget and to specification.
- Milestones are zero-duration checkpoints; deliverables are tangible products handed over.
- Risk management: identify, analyse, monitor, mitigate.
- Typical roles: project manager, analyst, designer, programmer, tester; communication and documentation standards hold the team together.
- Reuse (code, component, system levels) saves time and increases reliability but brings fitting, licensing and dependency problems.
- Maintenance — corrective, adaptive, perfective, preventive — dominates lifetime software cost; good documentation is the maintainer's best tool, especially for legacy systems.

7 INTEGRATION ACTIVITY

7.1 Aims of the activity

This integration activity closes the chapter by asking you to apply, in one coherent mini-project, everything you have learnt about software development approaches. It is a **guided rehearsal** of the complete software life cycle: from the first interview with the client to the maintenance contract, through specification, design, choice of tools, testing and project planning. By the end of the activity, you should be able to:

- choose a **life-cycle model** for a given project and **justify** that choice with arguments (requirement stability, risk, client availability, team size);
- conduct a short **requirements elicitation** interview and write **functional** and **non-functional requirements** in the format of a **Software Requirements Specification** (SRS);
- produce a level-0 **Data Flow Diagram** (DFD) and a simple **architectural design** diagram;
- select appropriate **development tools** (language, DBMS, CASE tools, version control) and justify each choice;
- design **test cases** (normal, boundary and erroneous data) for a critical module;
- plan a project with a **Gantt chart**, identify **risks** and propose mitigation measures;
- propose a **maintenance plan** covering corrective, adaptive and perfective maintenance;
- evaluate the software proposal of another team using a **quality checklist** (based on quality criteria such as correctness, reliability, usability and maintainability).

KEY POINTS **Skills mobilised**

Analysis and specification, design, choice of tools, verification and validation, project management, teamwork, maintenance and reporting: this activity is a complete rehearsal of the whole chapter, in the same spirit as the GCE Advanced Level problem-solving questions, where you must not only know the concepts but use them in a realistic situation and defend your choices.

7.2 Situation

EXPERIMENT **The Biyem-Assi Comprehensive High School canteen**

Biyem-Assi Comprehensive High School (Yaoundé) has 1 200 students. Its canteen sells about 600 meals per day at 500 FCFA each, paid in cash at a single counter. The canteen manager, Mrs Etoundi, faces serious problems:

- long queues: students lose up to 25 minutes of the lunch break, and some skip meals;
- cash handling causes regular shortages (about 3% of daily takings cannot be accounted for);
- the cook never knows in advance how many meals to prepare: on some days 100 meals are wasted, on others 80 students find nothing left;
- the bursar receives only a hand-written notebook as a daily report, often late and full of errors;
- parents complain that pocket money given for food is sometimes spent elsewhere.

The Principal decides to hire a software company to develop a Canteen Management System (CMS) with the following expectations:

- students book meals before 9:00 a.m. (via a school computer room terminal or a simple phone interface);
- payment is made by mobile money (MTN MoMo or Orange Money) into the school's account, or by a prepaid card credited by parents;
- the system produces an automatic daily report (meals booked, meals served, revenue, shortages) for the bursar before 4:00 p.m.;
- the cook receives the number of meals to prepare at 9:15 a.m.;
- the system must run on the school's two old computers (4 GB RAM, Windows 10) and be usable by staff with basic computer skills;
- budget: at most 1 500 000 FCFA; deadline: 3 months, before the second term.

Your class is divided into teams of 4–5 students. Each team plays the role of a software company competing for the contract. Your teacher plays the role of the client (the Principal) and can be interviewed.

METHOD **How to attack the mini-project**

1. Read the situation twice and highlight every problem and every constraint (budget, deadline, hardware, users).
2. Prepare the interview: write at least six questions for the client before meeting him/her (Who uses the system? What happens if the network fails? Who enters the data?).
3. Write the SRS first: every later artefact (DFD, architecture, tests) must be traceable to a numbered requirement.

4. Design before coding: draw the DFD and the architecture and check them against the SRS.
5. Test what is most critical: here, money — so the payment module gets the most test cases.
6. Plan and protect: schedule the work on a Gantt chart and list the risks with mitigations.
7. Review as a team: one member checks consistency between all documents before the presentation.

7.3 Tasks

Work in your team and produce a written file plus a 5–minute oral presentation.

1. **Choice of life-cycle model.** Choose one life-cycle model (waterfall, iterative/incremental, prototyping, spiral or agile) for this project. Justify your choice with **at least three arguments** linked to the situation (requirement clarity, client availability, risk, deadline, team experience). State one weakness of your chosen model for this project and how you will reduce it.
2. **Requirements analysis.** Interview the client (your teacher) for 10 minutes, then write:
   (a) **five functional requirements** (what the system shall do), numbered FR1–FR5;
   (b) **three non-functional requirements** (performance, usability, security, portability…), numbered NFR1–NFR3;

   using the SRS sentence pattern: "The system shall …" Each requirement must be unambiguous, testable and traceable to a problem in the situation.
3. **Design.** Draw:
   (a) a **level-0 DFD** showing the main processes (booking, payment, meal preparation forecast, reporting), the external entities (Student, Parent, Cook, Bursar, Mobile Money Operator) and the data stores;
   (b) a simple **architecture diagram** (e.g. two-tier or three-tier: presentation / application logic / database) naming the hardware and software components.
4. **Development tools.** List the tools your company would use: programming language, DBMS, diagramming/CASE tool, version control system, and testing tool. Justify each choice in one sentence, considering the school's old computers and the budget.
5. **Testing.** Write **six test cases** for the payment module in a table with columns: Test ID, Description, Input data, Expected result, Type (normal / boundary / erroneous). Cover at least: a successful payment, insufficient balance, a payment of 0 FCFA, a network failure during the transaction, and a duplicate payment attempt.
6. **Project management.** Draw a **Gantt chart** for the 3–month project (activities: requirements, design, coding, testing, deployment, training, documentation). Identify **two risks** (e.g. mo-

bile money API changes, power cuts, client unavailable) and give a mitigation measure for each.

7. **Maintenance plan.** Propose a maintenance plan covering the three types of maintenance (corrective, adaptive, perfective) with one concrete example of each for the CMS, plus who will maintain the system and at what cost.

8. **Presentation and evaluation.** Present your proposal in 5 minutes. The class evaluates each team using the quality checklist below (2 points per criterion, out of 20).

Criterion	Question asked	Mark /2
Correctness	Do the requirements match the client's real problems?	
Completeness	Are all seven tasks covered, with nothing essential missing?	
Consistency	Do the DFD, architecture and test cases agree with the SRS?	
Feasibility	Is the plan realistic for 3 months and 1 500 000 FCFA?	
Testability	Are the requirements and test cases precise and verifiable?	
Usability	Is the solution adapted to users with basic computer skills?	
Reliability	Are failures (network, power, cash shortage) handled?	
Maintainability	Is there a credible maintenance plan?	
Justification	Are the model and tool choices argued, not just stated?	
Presentation	Was the report clear, structured and within 5 minutes?	

7.4 Model answer

The following is a condensed model answer, at the level expected of a good Upper Sixth team. Read it after attempting the tasks yourself: compare it with your own proposal and note where your arguments were weaker or your documents less precise.

1. Choice of life-cycle model. We choose an **iterative and incremental model with prototyping**. Justification: (a) the requirements are only partly clear — the Principal knows the problems but not the exact screens, so a **prototype** of the booking interface will help elicit real needs; (b) the client (the school) is available for frequent feedback, which iterative development exploits; (c) the 3-month deadline is tight, so delivering the system in two increments (Increment 1: booking + meal forecast; Increment 2: mobile money payment + reports) guarantees that something useful is ready even if time runs out. Weakness: poor control of iterations can cause delays; we reduce this by fixing the content of each increment in advance and reviewing progress weekly with the client.

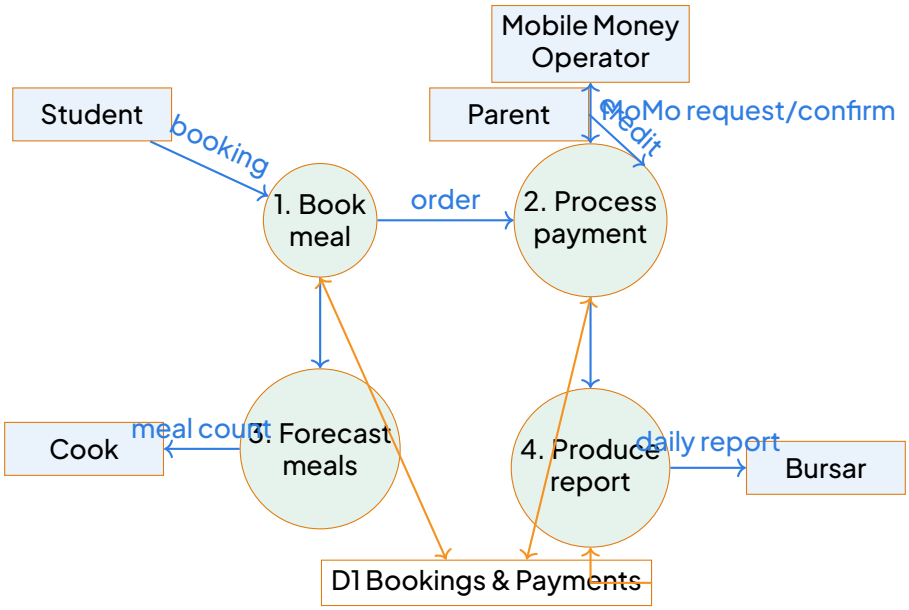
2. Requirements (SRS extract).

- **FR1.** The system shall allow a student to book at most one meal per day before 9:00 a.m.
- **FR2.** The system shall accept payment of 500 FCFA per meal by MTN MoMo, Orange Money or prepaid card, and shall issue an electronic receipt for each successful payment.
- **FR3.** The system shall send the total number of booked meals to the cook at 9:15 a.m. every school day.

- **FR4.** The system shall produce a daily report (meals booked, meals served, revenue, difference) printable by the bursar before 4:00 p.m.
- **FR5.** The system shall allow a parent to credit a student's prepaid card and to view the student's meal history.
- **NFR1 (performance).** The system shall process a booking or payment in at most 10 seconds, for up to 600 transactions per day.
- **NFR2 (usability).** The interface shall be in English and French and usable by canteen staff after at most 30 minutes of training.
- **NFR3 (security/reliability).** Every financial transaction shall be recorded in a log, and no booking shall be lost if power fails during the day (daily backup at minimum).

Notice how each requirement answers a problem of the situation: FR1 and FR3 solve the cook's uncertainty, FR2 and NFR3 attack the cash shortages, FR4 replaces the error-prone notebook, and FR5 answers the parents' complaint.

3. Design. Level-0 DFD:



Level-0 DFD of the Canteen Management System: rectangles are external entities, circles are processes, the open rectangle D1 is the data store.

Architecture: a **three-tier architecture** — (i) presentation tier: a simple web interface in English/French running in a browser on the two school computers; (ii) application tier: a PHP/Python application server hosted on one of the school computers; (iii) data tier: a MySQL database with daily backup to an external drive. The mobile money operator is reached through its web API over the school's internet connection. This architecture is cheap (no extra hardware), centralises the data (one consistent database for bookings and payments) and keeps the user machines almost empty, which suits the old computers.

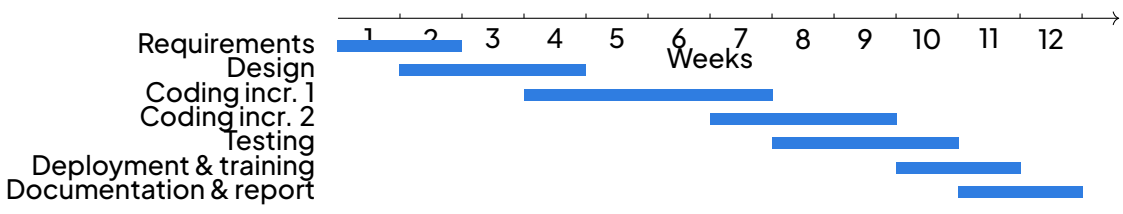
4. Development tools. Language: PHP (free, runs on old machines, many local developers can

maintain it). DBMS: MySQL (free, lightweight, well documented). Diagramming: diagrams.net (free CASE tool for DFDs and architecture diagrams). Version control: Git (free, keeps the full history of the code and enables teamwork without overwriting each other's work). Testing: manual test scripts plus PHPUnit for automating the payment module tests. All tools are free of licence fees, which respects the 1 500 000 FCFA budget.

5. Test cases for the payment module.

ID	Description	Input	Expected result	Type
T1	Normal MoMo payment	500 FCFA, valid number	Payment accepted, receipt issued, booking confirmed	Normal
T2	Insufficient balance	500 FCFA, balance 300 FCFA	Payment refused, clear message, no booking recorded	Erroneous
T3	Zero amount	0 FCFA	Rejected with "invalid amount" message	Boundary
T4	Network failure mid-transaction	500 FCFA, connection cut	Transaction rolled back, no double debit, retry offered	Erroneous
T5	Duplicate payment for same meal	Pay twice for same booking	Second payment blocked or refunded	Erroneous
T6	Maximum daily load	600 payments in one day	All processed in at most 10 s each, report correct	Boundary

6. Project management. Gantt chart (3 months, weeks 1–12):



Gantt chart of the 3-month CMS project (bars show activity durations; testing overlaps the second increment).

Risks: **R1:** the mobile money API or tariffs change during development — mitigation: isolate the payment code in one module and sign a written agreement with the operator early. **R2:** power cuts and internet failures at the school — mitigation: offline mode for booking with later synchronisation, and daily backups to an external drive.

7. Maintenance plan. Corrective maintenance: fix bugs found after deployment (e.g. a re-

ceipt not printed), free during a 3-month warranty, then covered by a yearly contract of 150 000 FCFA. Adaptive maintenance: update the system when its environment changes (e.g. a new meal price, a new MoMo tariff, or a Windows update). Perfective maintenance: add features requested later by users, such as an SMS notification to parents after each booking. The school computer teacher is trained for first-level maintenance (backups, user accounts); the company provides second-level support (code changes).

DID YOU KNOW? **Why mobile money changes everything**

In Cameroon, mobile money services such as MTN MoMo and Orange Money allow payments to be traced automatically: every transaction has a reference, a time and an amount. For a school canteen, this traceability is exactly what removes the "unexplained shortages" of a cash box — a concrete example of software improving accountability, not just speed.

WARNING **Common mistakes in this activity**

Do not write vague requirements ("the system shall be fast") — make them measurable ("at most 10 seconds"). Do not confuse functional requirements (actions the system performs) with non-functional ones (qualities and constraints). In the DFD, every process must have at least one input and one output flow, and data stores must not receive flows directly from external entities without a process in between. Finally, a Gantt chart without risks, or test cases covering only normal data, will lose marks at the GCE.

EXERCISE 7 **Going further**

Another school asks your team for a similar system, but this time the requirements are completely fixed in a signed document, the client will travel for the whole duration of the project, and the deadline is 6 months. (a) Which life-cycle model would you now choose, and why? (b) Which two activities of your plan would change the most compared with the CMS project?

SOLUTION **Exercise: Going further**

(a) The waterfall model becomes acceptable here: the requirements are fixed and signed (no need for prototyping or frequent feedback), and the client is unavailable, which would starve an iterative or agile approach of the feedback it depends on. The longer deadline also removes the pressure to deliver early increments. (b) Requirements analysis becomes shorter but must be extremely rigorous, since no clarification will be possible later; and verification and validation gains weight, because acceptance testing at the end is the only moment the client sees the product — so reviews and inspections must

replace continuous client feedback during development.

8 METHODS AND WORKED EXAMPLES

8.1 Method 1 — Choosing the Appropriate Life-Cycle Model

METHOD **Selecting a software development model for a given project**

Exam questions often describe a real project and ask you to propose and justify a development model. Proceed as follows:

1. Read the scenario carefully and underline: size of the project, clarity of requirements, availability of the client, risk level, need for early delivery, possibility of changes, regulatory constraints.
2. Recall the key features of each model:
   - **Waterfall**: sequential phases (requirements → analysis → design → coding → testing → deployment → maintenance); requirements must be complete, stable and well understood; little client involvement after requirements sign-off; documentation-heavy.
   - **V-model**: extension of waterfall where each development phase has a corresponding testing phase planned in parallel (unit tests from detailed design, integration tests from architectural design, system tests from requirements specification, acceptance tests from user requirements). Emphasises early test preparation and verification/validation.
   - **Prototyping**: a quick working model (throw-away or evolutionary) is built, shown to the client, refined repeatedly; ideal when requirements are vague or user interface is critical.
   - **Incremental**: the system is delivered in small functional versions (increments); each increment adds features; requirements for early increments must be stable.
   - **Iterative / Spiral**: cycles of plan – analyse risks – build (or prototype) – evaluate; each cycle produces a more complete version; best for large, high-risk, expensive projects where risk management is paramount.
   - **Agile** (e.g. Scrum, XP): short iterations (sprints, 2–4 weeks), continuous client collaboration, welcomes changing requirements, working software over comprehensive documentation, self-organising teams.
   - **Rapid Application Development (RAD)**: heavy use of prototyping, time-boxing, component reuse, and active user involvement to deliver quickly; suited for projects with tight deadlines and modular requirements.
3. Match the scenario to the model: vague requirements → prototyping, agile or RAD; fixed contract with stable requirements → waterfall or V-model; high risk → spiral; need for early partial delivery → incremental/agile; strict regulatory testing → V-model.

4. Justify your choice with at least two arguments drawn directly from the scenario, and state one limitation of the model you rejected — this earns full marks.

WORKED EXAMPLE 7 **Choosing a model for a school results system**

The MINEDUB regional delegation in Bamenda wants a system to process end-of-term results for 40 secondary schools. The head of the project says: "We know exactly what we want: input marks, compute averages and ranks, print report cards. The rules for computing averages were fixed by an official text ten years ago and will not change. The system must be ready in eight months and thoroughly tested before any school uses it."

a) Propose a suitable development model.

b) Justify your choice with two arguments.

c) State one reason why the agile approach would be less appropriate here.

SOLUTION

a) Model proposed: the **V-model** (an enhanced waterfall model with integrated test planning).
b) Justification.

- Requirements are complete and stable. The client states "we know exactly what we want" and the computation rules were fixed ten years ago and "will not change". A sequential model is therefore safe: there is no need for prototyping or iterative refinement of requirements.
- Testing is critical and must be planned early. The system will process official results for 40 schools and "must be thoroughly tested before any school uses it". In the V-model, every development phase corresponds to a test phase prepared in advance: acceptance tests are derived from user requirements, system tests from the requirements specification, integration tests from the architectural design, and unit tests from the detailed design. This guarantees a rigorous validation before deployment, which matches the client's demand for thorough testing.

c) Why agile is less appropriate. Agile methods rely on frequent delivery of partial versions and on requirements that evolve sprint after sprint with continuous client feedback. Here the requirements are frozen by an official regulation, so the main strength of agile (welcoming change) brings no benefit, while delivering unfinished versions of an examination-results system to schools would be unacceptable and risky. Moreover, the eight-month deadline with a fixed scope is better managed by a plan-driven approach.

8.2 Method 2 — Analysing and Classifying Requirements

METHOD **Writing a requirements specification (SRS)**

1. Identify the stakeholders (client, end-users, administrators, regulators) and gather needs by interviews, questionnaires, observation, document study and workshops.
2. Classify each requirement according to the ISO/IEC 25010 quality model:
   - **Functional requirement**: what the system shall do — a service, a reaction to an input, a computation. Example: "The system shall compute each candidate's weighted average using the official formula."
   - **Non-functional requirement**: a quality attribute or constraint: performance (response time, throughput), security (confidentiality, integrity), usability, reliability (availability, fault tolerance), maintainability, portability, compatibility, legal/regulatory compliance.
3. Write each requirement so that it is (IEEE 830 characteristics): unambiguous (one interpretation only), complete (no missing information), verifiable/testable (measurable pass/fail criterion), consistent (no conflict with other requirements), traceable (uniquely numbered, e.g. REQ-F-01, REQ-NF-05), and feasible.
4. Prioritise (essential / desirable / optional — or MoSCoW: Must, Should, Could, Won't) and get the client's formal validation: the signed requirements specification is the baseline contract between client and developer.
5. Reflex: if a requirement contains a number (time, size, percentage, availability %), it is usually non-functional; if it contains a verb of action done by the system (shall compute, shall display, shall send), it is functional.

WORKED EXAMPLE 8 **Classifying requirements for a mobile money application**

A microfinance institution in Douala wants a mobile application. The director lists the following wishes:

(1) "A customer must be able to transfer money to another customer."

(2) "Every transaction must be encrypted."

(3) "The application must display the account balance after each operation."

(4) "The application must work on Android version 8 and above."

(5) "A transfer must be confirmed by a PIN code."

(6) "The system must handle 500 transactions per minute."

Classify each wish as a functional (F) or non-functional (NF) requirement, justifying your answer, and rewrite wish (6) so that it is testable.

SOLUTION

Classification.

- (1) F — it describes a service the system provides (transferring money): an action performed by the system in response to a user request.
- (2) NF — encryption is a security quality attribute (confidentiality/integrity) imposed on all transactions, not a user-visible service on its own.
- (3) F — displaying the balance is a reaction of the system to an operation; it is an observable behaviour (output).
- (4) NF — this is a portability/compatibility constraint on the execution environment, not a function.
- (5) F — requiring PIN confirmation is a precise behaviour of the system during a transfer (a functional security rule). Note: the general need "the system must be secure" would be NF, but a concrete confirmation mechanism is F.
- (6) NF — it expresses a performance quality (throughput capacity), not a service.

Making (6) testable. A testable requirement must be measurable, verifiable and unambiguous:

> REQ-NF-06: Under a sustained load of 500 transfer requests per minute generated by the standard benchmark tool, the system shall process at least 495 transactions per minute with zero transaction loss and a mean response time not exceeding 3 seconds per transaction.

This version is quantified (495/min, 3 s), verifiable (a load test can check it) and unambiguous (conditions of measurement are stated: sustained load, standard benchmark, zero loss).

8.3 Method 3 — Software Design: Stages, Notations and Documentation

METHOD **The software design process and its deliverables**

Design transforms the what (requirements) into the how (architecture and detailed logic). It proceeds in two main stages:

1. Architectural (High-Level) Design: decomposes the system into major subsystems/modules, defines their responsibilities, interfaces, data flows, and global data structures. Key decisions: architectural style (layered, client-server, MVC, microservices), technology stack, database schema outline, security architecture.
2. Detailed (Low-Level) Design: specifies the internal logic of each module — algo-

rithms, data structures, class diagrams (attributes, methods, visibility), state machines for reactive objects, and precise interface contracts (pre/post-conditions).

Standard notations (GCE expects familiarity with):

- **Flowcharts** / **Structure charts** (Jackson) for algorithmic logic and module hierarchy.
- **Data Flow Diagrams (DFD)** — context diagram (level 0) and levelled decomposition for functional view.
- **Entity-Relationship Diagrams (ERD)** for persistent data design.
- **UML basics**: Use Case diagram (requirements), Class diagram (static structure), Sequence diagram (interaction), State diagram (behaviour).

Design documentation (IEEE 1016): Software Design Description (SDD) containing: system overview, architectural design, detailed design for each module, data dictionary, interface specifications, traceability matrix linking design elements to requirements.

METHOD **Designing and verifying an algorithm with a flowchart**

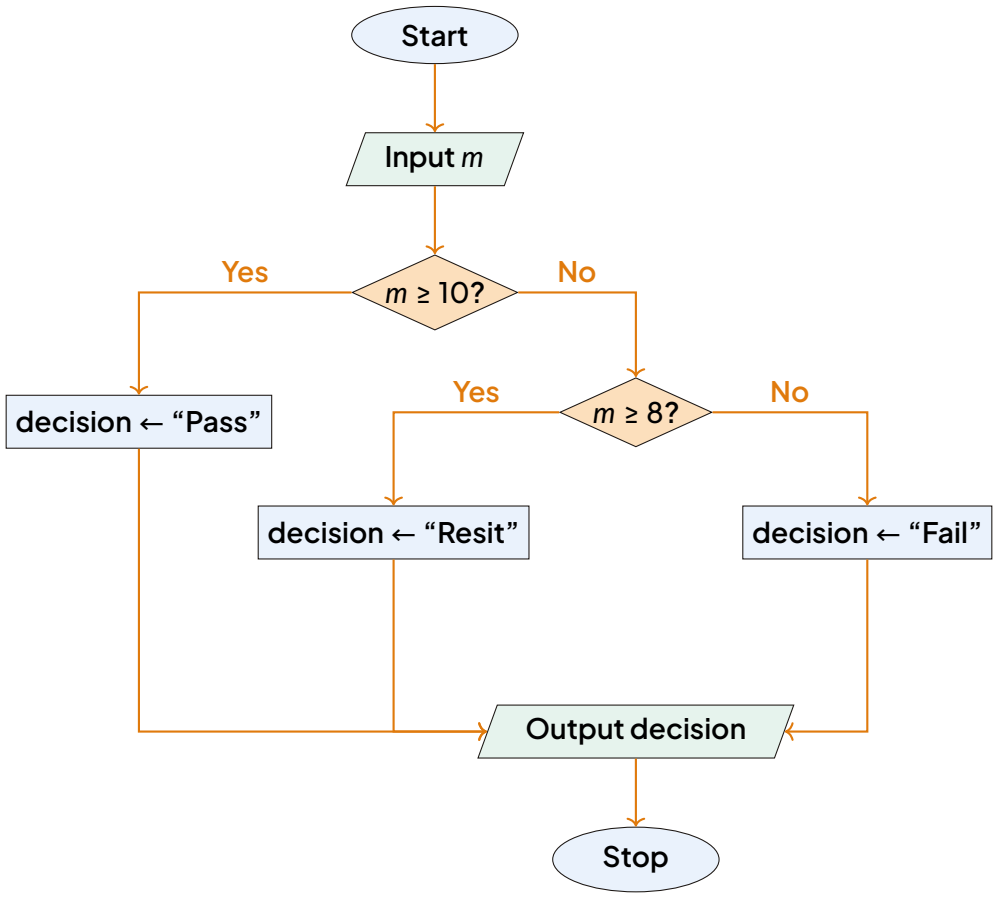
1. Identify inputs, outputs and the processing steps from the specification.
2. Draw the flowchart using standard ISO 5807 symbols: oval (start/stop), parallelogram (input/output), rectangle (processing/assignment), diamond (decision), arrows for flow of control.
3. Check the logic: every decision diamond has exactly two exits (Yes/No or True/False); every path must eventually reach Stop; no unreachable code.
4. Validate with a trace table: choose representative test values (normal case, boundary case, special/invalid case) and record the value of every variable after each step.
5. Reflex: a trace table is the paper equivalent of a debugger; at the GCE it is the standard way to prove that a design is correct before coding.

WORKED EXAMPLE 9 **Flowchart and trace table for a grading algorithm**

A school uses the following rule to grade a mark m out of 20: if $m \geq 10$ the decision is "Pass", otherwise if $m \geq 8$ the decision is "Resit", otherwise "Fail". Draw the flowchart of the algorithm and build a trace table for the inputs $m = 14$, $m = 9$, $m = 5$.

SOLUTION

Flowchart.



Flowchart of the grading algorithm.

Trace table. We follow the variables *m* and decision:

Step	*m*	decision	Comment
Input	14	–	read mark
m ≥ 10?	14	–	true
assign	14	Pass	first branch
Input	9	–	read mark
m ≥ 10?	9	–	false
m ≥ 8?	9	–	true
assign	9	Resit	second branch
Input	5	–	read mark
m ≥ 10?	5	–	false
m ≥ 8?	5	–	false
assign	5	Fail	third branch

The three outputs (Pass, Resit, Fail) match the specification, so the design is validated. Note the importance of testing the boundary values: $m = 10$ must give Pass and $m = 8$ must give Resit — a common error is to write $m > 10$ instead of $m \geq 10$.

8.4 Method 4 — Software Development Tools (Lesson 31)

METHOD **Categories of CASE tools and development utilities**

Software development tools (Computer-Aided Software Engineering — CASE) support the entire life cycle. Know the main categories:

1. Requirements/Analysis tools: requirements management (traceability, versioning), modelling (DFD, ERD, UML editors).
2. Design tools: architecture/design modellers (UML class, sequence, state diagrams), code generators from models.
3. Programming tools: IDE (Integrated Development Environment) — editor with syntax highlighting, code completion, refactoring, build automation; compilers/interpreters; debuggers (breakpoints, watch variables, call stack); static analysers (lint, SonarQube) for coding standards and bug patterns.
4. Testing tools: test management (test cases, plans, traceability), unit test frameworks (JUnit, pytest), coverage analysers (statement, branch, path coverage), load/performance tools (JMeter), automation frameworks (Selenium).
5. Configuration Management (SCM) tools: version control (Git, SVN) — repository, branching/merging, tagging, conflict resolution; build/CI tools (Maven, Gradle, Jenkins, GitHub Actions) for continuous integration.
6. Project management tools: scheduling (Gantt, PERT), issue/bug tracking (Jira, Bugzilla), documentation wikis.
7. Maintenance/Reengineering tools: reverse engineering (code to model), refactoring browsers, complexity metrics (cyclomatic complexity, Halstead).

Upper CASE (front-end): analysis/design. Lower CASE (back-end): code generation, compilation, testing. Integrated CASE (I-CASE): seamless repository sharing across phases.

WORKED EXAMPLE 10 **Selecting tools for a student project**

A team of four Upper Sixth students must build a web-based library system in 10 weeks using Python/Django and PostgreSQL. They need to manage requirements, code ver-

sions, automated tests, and continuous deployment to a test server. Propose a toolchain (one tool per category) and justify each choice.

SOLUTION

Category	Tool	Justification
Version control	Git + GitHub	Distributed, free, branching for features, pull requests for code review
Issue tracking	GitHub Issues / Trello	Integrated with repo, lightweight, Kanban board for sprints
CI/CD	GitHub Actions	Free for public repos, YAML workflows, runs tests on every push
Testing	pytest + coverage.py	Standard Python, fixtures, parameterised tests, coverage reports
Static analysis	flake8 / pylint	Enforces PEP 8, catches bugs early, runs in CI
Documentation	MkDocs / Sphinx	Markdown/reST, auto-deploy to GitHub Pages

Key point: the toolchain must be integrated (e.g. push → CI runs tests → coverage gate → deploy) so that quality feedback is immediate.

8.5 Method 5 — Designing Test Cases (Verification, Validation and Testing)

METHOD **Building a test plan and test cases**

1. Distinguish the vocabulary:
   - **Verification**: "Are we building the product right?" — checks each phase against its specification (reviews, inspections, walkthroughs, trace tables, static analysis).
   - **Validation**: "Are we building the right product?" — checks the final system against the user's needs (acceptance tests, beta testing).
2. Choose the test levels in order (V-model alignment):
   - Unit testing (white-box, developer): each module/class in isolation; stubs/drivers for dependencies.
   - Integration testing (grey-box): modules combined — top-down, bottom-up, sandwich; tests interfaces and data flow.
   - System testing (black-box, QA team): whole product against requirements spec-

ification — functional, performance, security, usability, compatibility.

- Acceptance testing (black-box, client/users): alpha (in-house), beta (real environment); validates fitness for purpose.

3. Design test cases using two complementary strategies:
   - Black-box (specification-based): derive cases from requirements without seeing code — equivalence partitioning (valid/invalid classes), boundary value analysis (edges of classes), decision tables, state transition testing.
   - White-box (structure-based): derive cases from code to achieve coverage criteria — statement coverage, branch/decision coverage, path coverage (limited to feasible paths).
4. For each test case, record: unique identifier, test objective, preconditions, input data, expected result (computed from spec, not from code), actual result, verdict (pass/fail), traceability to requirement. A test without an expected result is useless.
5. Reflex: always include at least one normal, one boundary and one invalid/abnormal input per equivalence class.

WORKED EXAMPLE 11 **Black-box test cases for a fee calculation module**

A school-fees module computes the discount on annual fees of 150 000 FCFA according to the number n of children from the same family enrolled: if $n \leq 1$ no discount; if $2 \leq n \leq 3$ a discount of 10%; if $n \geq 4$ a discount of 20%. The module must reject any $n < 1$ with an error message.

a) Identify the equivalence classes of the input n.

b) Give a black-box test table with at least five test cases, including boundary values, with expected results.

c) Explain why these tests are called “black-box” tests.

SOLUTION

a) Equivalence classes. From the specification, the input domain splits into four classes:

- C_1: $n < 1$ (invalid) → error message;
- C_2: $n = 1$ → discount 0%, amount 150 000 FCFA;
- C_3: $2 \leq n \leq 3$ → discount 10%, amount 135 000 FCFA;
- C_4: $n \geq 4$ → discount 20%, amount 120 000 FCFA.

The critical boundaries are $n = 0, 1, 2, 3, 4$.
b) Test table.

Test ID	Input n	Expected result	Class / Boundary
T1	0	Error message "invalid number"	C_1, upper boundary (invalid)
T2	1	Amount = 150 000 FCFA	C_2, lower/upper boundary
T3	2	Amount = 135 000 FCFA	C_3, lower boundary
T4	3	Amount = 135 000 FCFA	C_3, upper boundary
T5	4	Amount = 120 000 FCFA	C_4, lower boundary
T6	7	Amount = 120 000 FCFA	C_4, normal value

Computations: 150 000 × (1 − 0.10) = 135 000; 150 000 × (1 − 0.20) = 120 000.
c) Why "black-box". The cases were derived only from the specification (the discount rules), without any knowledge of the program's internal code. The module is treated as a closed box: we stimulate the inputs and observe whether the outputs match the expected results. Had we instead read the code to make sure every `if` branch is executed, we would be doing white-box testing. The two approaches are complementary: black-box tests detect missing or wrong functionality, white-box tests detect hidden logic errors in rarely executed branches.

8.6 Method 6 — Project Management: Scheduling, Risk and Configuration Management

METHOD **Scheduling tasks, critical path and risk management**

1. List the tasks with their durations and their predecessors (tasks that must finish before a task can start). Build a precedence table.
2. Compute earliest times (forward pass): earliest start (ES) = max of predecessors' earliest finishes (EF); EF = ES + duration. The earliest finish of the last task(s) gives the minimum project duration.
3. Compute latest times (backward pass): latest finish (LF) of final task = project duration; latest start (LS) = LF − duration; for predecessors, LF = min of successors' LS. Slack (float) = LS − ES (or LF − EF).
4. Critical path = chain of tasks with zero slack; any delay on it delays the whole project. Near-critical tasks (small slack) need monitoring.
5. Draw the Gantt chart: one horizontal bar per task, length = duration, positioned at its earliest start; critical tasks highlighted. Show milestones (diamonds).
6. Risk management: identify risks (probability × impact), plan mitigation (avoid, reduce, transfer, accept), monitor triggers. Common software risks: staff turnover, requirement creep, technology failure, estimation errors.

7. Configuration Management (CM): identify configuration items (code, docs, tests), version control, change control board (CCB), baseline management, build/release procedures.
8. Reporting: progress reports (weekly), milestone reviews, burn-down charts (agile), earned value (planned vs actual cost/schedule).

WORKED EXAMPLE 12 **Critical path and risk analysis for a pharmacy stock system**

The development of a stock-management application for a Yaoundé pharmacy is decomposed as follows:

Task	Description	Duration (weeks)	Predecessors
A	Requirements analysis	2	–
B	Design	3	A
C	Database coding	4	B
D	Interface coding	2	B
E	Integration and testing	2	C, D
F	Deployment and training	1	E

a) Compute the earliest start and finish of each task and the minimum project duration.

b) Determine the critical path and the slack of each task.

c) The interface coder falls ill and task D takes 4 weeks instead of 2. Does the project end date change? Justify.

d) Identify one major risk in this project and propose a mitigation strategy.

SOLUTION

a) Forward pass. ES = max of predecessors' EF; EF = ES + duration.

Task	Duration	ES	EF	Predecessors
A	2	0	2	–
B	3	2	5	A
C	4	5	9	B
D	2	5	7	B
E	2	9	11	C, D
F	1	11	12	E

For E: ES = $\max(EF_C, EF_D) = \max(9, 7) = 9$. The minimum project duration is 12 weeks.

b) Backward pass and slack. LF of F = 12; LS = 11. LF of E = 11; LS = 9. LF of C = 9; LS = 5. LF of D = 9; LS = 7. LF of B = 5; LS = 2. LF of A = 2; LS = 0.

Task	ES	EF	LS	LF	Slack
A	0	2	0	2	0
B	2	5	2	5	0
C	5	9	5	9	0
D	5	7	7	9	2
E	9	11	9	11	0
F	11	12	11	12	0

Critical path: $A \to B \to C \to E \to F$ (zero slack). Task D has slack = 2 weeks.
c) Effect of D's delay. If D takes 4 weeks, $EF_D = 5 + 4 = 9$. ES of E = $\max(9, 9) = 9$, unchanged. The project still finishes in 12 weeks, because the delay (+2 weeks) equals D's slack. However, D now has zero slack and becomes critical: any further delay on D would postpone delivery. This illustrates why a project manager monitors tasks whose slack is shrinking.
d) Risk and mitigation. Risk: "Key developer (database coder) falls ill during task C (4 weeks, on critical path)." Probability: medium; Impact: high (delays critical path). Mitigation: pair programming / knowledge sharing during task B so another developer can take over; document database schema and APIs thoroughly; maintain a backup developer on standby.

8.7 Method 7 — Evaluating and Acquiring Software

METHOD **Evaluating a software product or an acquisition option**

1. Define evaluation criteria before looking at products, based on ISO/IEC 25010 (product quality) and project context:
   - Functional suitability (completeness, correctness, appropriateness)
   - Reliability (maturity, availability, fault tolerance, recoverability)
   - Performance efficiency (time behaviour, resource utilisation, capacity)
   - Usability (learnability, operability, accessibility)
   - Security (confidentiality, integrity, authenticity, accountability)
   - Maintainability (modularity, reusability, analysability, testability)
   - Portability (adaptability, installability, replaceability)

   Plus cost (licence, hardware, training, maintenance), vendor viability/support, com-

patibility with existing infrastructure, licensing model (proprietary, open source, SaaS).

2. Compare acquisition options:
   - Off-the-shelf (COTS): cheap, immediate, tested by many users; but may not fit exactly, creates vendor lock-in, upgrade dependency.
   - Custom (bespoke) development: fits requirements exactly, competitive advantage, full control; but expensive, slow, high risk, maintenance burden.
   - Open source: no licence fee, source code modifiable, community innovation; but support and long-term maintenance must be organised, integration effort, licence compliance (GPL, MIT, Apache).
   - SaaS / Cloud: low upfront cost, scalable, vendor maintains infrastructure; but data sovereignty, recurring cost, limited customisation, internet dependency.
3. Score and decide: build a weighted grid (criterion × weight × mark / max), total the scores, and justify the final choice by the highest-priority criteria of the scenario, not only by the arithmetic total.
4. Reflex: never evaluate software on price alone — the GCE expects a multi-criteria argument linked to the organisational context.

WORKED EXAMPLE 13 **Weighted evaluation of two library systems**

A teachers' training college in Buea must choose between two library management systems. The priorities are: functionality (weight 4), ease of use (weight 3), cost (weight 2), local technical support (weight 1). Marks are out of 5.

Criterion	Weight	Product X	Product Y
Functionality	4	4	3
Ease of use	3	2	5
Cost	2	2	4
Local support	1	5	2

Compute the weighted score of each product and recommend one, justifying your answer.

SOLUTION

Weighted score of X:

$$S_X = 4(4) + 3(2) + 2(2) + 1(5) = 16 + 6 + 4 + 5 = 31.$$

Weighted score of Y:

$$S_Y = 4(3) + 3(5) + 2(4) + 1(2) = 12 + 15 + 8 + 2 = 37.$$

Recommendation: Product Y (37 against 31).
Justification. Although X is stronger on functionality (the heaviest criterion, weight 4) and on local support, Y dominates on ease of use (weight 3) and cost (weight 2), which together weigh 5. For a college library, the daily users are librarians and student teachers with varied computer skills: a system that is easy to use reduces training time and data-entry errors, and its lower cost frees budget for books and computers. The weakness of Y on local support (mark 2, weight 1) is the least penalising because it carries the smallest weight; it can be mitigated by negotiating a maintenance contract or training an in-house technician.
Examiner's remark: a candidate who simply recommends X "because functionality matters most" ignores the weights; a candidate who only quotes the totals without linking them to the college's context loses the justification mark. Both steps — computation and contextual argument — are required.

8.8 Method 8 — Diagnosing Maintenance Needs, Organising Reuse and Teamwork

METHOD **Classifying maintenance and advising on reuse/teamwork**

1. Classify the maintenance request into one of the four ISO/IEC 14764 types:
   - **Corrective**: fixing faults/bugs discovered after delivery (reactive).
   - **Adaptive**: modifying the software because its environment changed (new OS, new law, new hardware, new API).
   - **Perfective**: improving performance, maintainability or adding features requested by users (proactive enhancement).
   - **Preventive**: restructuring, documenting, refactoring or updating documentation to avoid future problems (technical debt reduction).
2. Ask the diagnostic question: "Why must the software change?" — because it is wrong (corrective), because the world around it changed (adaptive), because users want more/better (perfective), because the code is ageing/fragile (preventive).
3. For reuse questions: identify what can be reused (code libraries, components, frameworks, designs, test cases, requirements), and state the conditions for successful reuse: good documentation (clear contracts), standard interfaces, licensing compatibility, version stability, test suites, organisational support (reuse repository, librarian). Reuse reduces cost, development time and increases reliability (reused components are already tested in production).

4. For teamwork questions: mention the need for a project manager, clear role allocation (analyst, designer, programmer, tester, DevOps), communication tools (version control with branching strategy — Gitflow, trunk-based; issue tracker; code reviews; CI pipeline; documentation standards; daily stand-ups) so that several developers can work on the same system without conflicts. Cite Brooks' Law: "Adding manpower to a late software project makes it later" — communication overhead grows quadratically.

WORKED EXAMPLE 14 **Maintenance scenarios at a water distribution company**

The Cameroon Water Utilities corporation uses a billing application written several years ago. For each situation below, state the type of maintenance required and justify:

(1) Customers complain that bills for the month of February are computed with 30 days instead of 28/29.

(2) A new government decree changes the VAT rate from 19.25% to a new value; the application must apply it.

(3) The commercial department asks for a new feature: sending bills by SMS.

(4) Before adding the SMS feature, the programmers rewrite the tangled billing module into clean, documented functions, without changing what it does.

(5) The IT manager proposes to reuse the SMS-sending component already developed for the SMS feature in a future "water-cut alert" application. Give two conditions for this reuse to succeed and one benefit.

SOLUTION

- (1) Corrective maintenance. The program produces a wrong result (a bug in the day-count logic for February). The software itself is faulty; nothing external changed. The fault must be located, corrected, and the module re-tested — ideally with the boundary cases of February in leap and non-leap years (28, 29 days).
- (2) Adaptive maintenance. The software was correct, but its environment (the tax regulation) changed. The application must be adapted so that the VAT rate can follow the new decree — good practice is to make the rate a configurable parameter rather than a constant buried in the code, so that the next change needs no reprogramming.
- (3) Perfective maintenance. The system works correctly, but users request a new functionality (SMS billing) to improve the service. This is an enhancement of the product, not a repair.
- (4) Preventive maintenance. The behaviour of the module is unchanged; the code is restructured and documented to make future changes easier and safer. This is pre-

cisely the aim of preventive maintenance: reducing the cost and risk of tomorrow's interventions (here, the upcoming perfective change).

- (5) Reuse. Conditions: (i) the SMS component must be well documented (clear description of its inputs, outputs, pre/post-conditions, exceptions) so that another team can use it without reading all its code; (ii) it must have a standard, well-defined interface (e.g. a function `sendSMS(number, message)` returning a status code) independent of the billing context, and its licence must permit reuse. Benefit: development time and cost are reduced, and reliability is increased because the component has already been tested and used in production — the "water-cut alert" application can be delivered faster and with fewer faults.

8.9 Method 9 — Integration Activity: Putting It All Together (Lesson 36)

EXPERIMENT **Mini-project synthesis: from requirements to tested increment**

The integration activity (Lesson 36) requires you to demonstrate the complete cycle on a small case study. Typical GCE task: "Given a brief scenario, produce: (a) a requirements specification (functional + non-functional), (b) a design (flowchart or DFD + data dictionary), (c) a test plan with black-box test cases, (d) a project schedule (Gantt + critical path), (e) a maintenance classification for a given change request."
Worked outline for a "School Club Membership" system:

1. Requirements (extract):
   - REQ-F-01: The system shall register a student (name, class, club) and assign a unique membership ID.
   - REQ-F-02: The system shall record attendance at club meetings.
   - REQ-NF-01: The system shall respond to a registration request within 2 seconds (performance).
   - REQ-NF-02: Only the club coordinator shall delete a membership (security/access control).
2. Design: Context DFD (external entities: Student, Coordinator; process: Manage Membership; data stores: Members, Attendance). Flowchart for "Register Student" (input details → validate → generate ID → store → output ID). Data dictionary entries for Student, Membership, Attendance.
3. Test Plan: Unit tests for ID generation (boundary: first ID, max ID), integration test for registration+attendance link, system test for coordinator-only delete (attempt by student → reject).
4. Schedule: Tasks: Requirements (1 wk), Design (1 wk), DB + Backend (2 wks), Frontend (1 wk), Integration Test (1 wk), Deploy (0.5 wk). Critical path: Requirements → Design

→ Backend → Integration → Deploy = 5.5 wks. Frontend has 0.5 wk slack.

5. Maintenance: Six months later, the school changes the ID format from numeric to alphanumeric (e.g. "SC-2025-001"). This is adaptive maintenance (environment/business rule changed).

Examiner tip: keep each artefact concise but traceable. Number requirements, link design elements to requirement IDs, link test cases to requirements. Traceability is the hallmark of a disciplined process.

KEY POINTS **Reflexes for the examination**

- Model choice: vague requirements → prototyping/agile/RAD; stable requirements + strong testing mandate → V-model; high risk/large scale → spiral; early partial delivery → incremental/agile.
- A requirement must be unambiguous, verifiable, numbered, traceable; numbers (time, size, %) usually signal a non-functional requirement.
- Design: two stages (architectural + detailed); notations: flowchart, DFD, ERD, UML basics; validate every algorithm with a trace table (normal, boundary, invalid).
- Test levels: unit → integration → system → acceptance. Black-box (equivalence classes, boundaries) + white-box (statement/branch coverage). Every test case needs an expected result.
- Tools: know categories (requirements, design, IDE/debugger, test, version control/CI, project management). Integrated toolchain = fast feedback.
- Project management: forward/backward pass for critical path; slack = float; monitor near-critical tasks. Risk = probability × impact; mitigate proactively. CM = version control + change control + baselines.
- Evaluation: multi-criteria weighted grid (ISO 25010 qualities + cost + support); justify with context, not just totals.
- Maintenance: wrong → corrective; environment changed → adaptive; users want more → perfective; code ageing → preventive.
- Reuse: needs documentation, standard interfaces, licence clearance, tests. Benefit: cost/time/reliability.
- Teamwork: roles, Git branching, code reviews, CI, communication. Brooks' Law: adding people to late project delays it further.
- Integration activity: demonstrate traceability from requirements → design → test → schedule → maintenance classification.

9 EXERCISES

9.1 I check my knowledge

EXERCISE 8 **MCQ: Life-Cycle Models**

Which of the following statements best describes the Waterfall model?

1. It allows for overlapping phases and frequent customer feedback.
2. It is iterative and delivers working software in short cycles called sprints.
3. It follows a strict linear sequence of phases where each phase must be completed before the next begins.
4. It focuses exclusively on prototyping to refine requirements.

EXERCISE 9 **True/False: Requirements Classification**

State whether the following statements are True or False. Justify your answer in each case.

1. "The system shall allow a librarian to issue a book to a student" is a non-functional requirement.
2. "The system response time for any query shall not exceed 2 seconds" is a functional requirement.
3. A Software Requirements Specification (SRS) document should be unambiguous, complete, and verifiable.
4. Functional requirements describe how the system performs its functions.

EXERCISE 10 **Definitions: Key Concepts**

Define the following terms precisely:

1. Software Verification
2. Software Validation
3. Corrective Maintenance
4. Perfective Maintenance

5. Critical Path (in Project Management)

EXERCISE 11 **Direct Application: Testing Levels**

Match each testing level in Column A with its correct description in Column B.

Column A	Column B
1. Unit Testing	A. Testing the entire system against the SRS to ensure it meets user needs.
2. Integration Testing	B. Testing performed by end-users in their own environment before final acceptance.
3. System Testing	C. Testing individual modules or components in isolation.
4. Acceptance Testing	D. Testing the interfaces and interaction between integrated modules.

9.2 I practise

EXERCISE 12 **Functional vs Non-Functional Requirements**

A hospital is developing a Patient Records Management System. The following requirements have been elicited. Classify each as Functional (F) or Non-Functional (NF) and provide a brief justification for two of your choices.

1. The system shall allow doctors to view a patient's medical history.
2. The system shall encrypt all patient data at rest and in transit using AES-256.
3. The system shall generate a monthly report of admitted patients per ward.
4. The system shall be accessible via web browsers on Windows, macOS, and Linux.
5. The system shall support up to 500 concurrent users without performance degradation.
6. The system shall send an SMS alert to a doctor when a critical lab result is recorded.
7. The database shall be backed up automatically every 24 hours.
8. The user interface shall be available in English and French.

EXERCISE 13 **Data Flow Diagram – Context & Level 0**

Draw a Context Diagram (Level 0 DFD) and a Level 1 DFD for an Online Exam Registration System with the following description:

- External Entities: Student, Examination Officer, Payment Gateway (Bank).
- Processes:
  1. Validate Registration: Checks student details and exam eligibility.
  2. Process Payment: Interacts with Payment Gateway to confirm fees.
  3. Assign Centre: Allocates an examination centre based on student location.
  4. Generate Slip: Produces the examination registration slip.
- Data Stores: Student Database, Exam Schedule Database, Registration Records.

Show all data flows clearly. Use standard DFD notation (Gane & Sarson or Yourdon).

EXERCISE 14 **Test Plan Design**

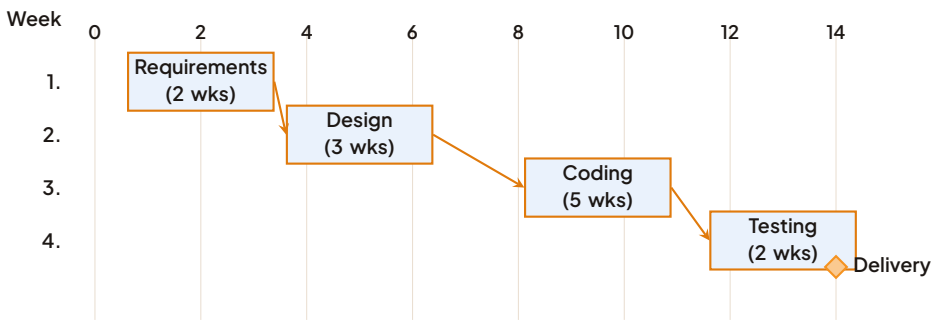
Design a test plan for a function `computeGrade(average)` that takes a student's numerical average (0–100) and returns a letter grade based on the following scale:

- A: 70–100
- B: 60–69
- C: 50–59
- D: 40–49
- F: 0–39
- Invalid input (negative or > 100): Return "Error".

Provide at least six test cases covering normal boundaries, invalid boundaries, and mid-range values. Present your answer in a table with columns: Test Case ID, Input (Average), Expected Output, Type of Testing (Boundary/Normal/Error).

EXERCISE 15 **Gantt Chart Interpretation**

The simplified Gantt chart below represents a software project schedule (durations in weeks).



Answer the following:

1. What is the total planned duration of the project (in weeks)?
2. Identify the critical path.
3. If the Design phase overruns by 2 weeks, what is the impact on the project end date? Explain.
4. Which phase has the longest duration?

9.3 I prepare for the GCE

EXERCISE 16 **Essay: Development Approach Selection – 15 marks**

A startup company in Douala wants to develop a Mobile Agricultural Marketplace App connecting farmers directly to buyers. The requirements are not fully known initially; the market changes rapidly, and the company needs to release a Minimum Viable Product (MVP) within 3 months to secure funding. User feedback will drive subsequent features.

1. Compare the Waterfall and Agile (Scrum) approaches for this project. Structure your comparison around: Requirements volatility, Delivery timeline, Customer involvement, and Risk management. (8 marks)
2. Recommend the most suitable approach. Justify your recommendation with three distinct arguments linked to the scenario. (4 marks)
3. Describe two key practices of the recommended approach that would ensure the MVP is delivered on time. (3 marks)

EXERCISE 17 **Case Study: Acquisition Strategy & Maintenance – 15 marks**

The Ministry of Secondary Education requires a new National Timetabling Application for all public secondary schools. The ICT department proposes three options:

- Option A: In-house development by the Ministry's IT team.
- Option B: Purchase a Commercial Off-The-Shelf (COTS) timetabling package from a foreign vendor.
- Option C: Adapt an existing Open Source timetabling system (e.g., FET) with a local contractor.

1. Evaluate each option against the following criteria: Total Cost of Ownership, Time to Deployment, Customisation to Cameroon's specific rules (e.g., bilingual system, shift schools), and Long-term Vendor Lock-in / Sustainability. (9 marks)
2. Recommend the best option for the Ministry. Justify your choice. (3 marks)
3. After deployment, the following maintenance requests arrive. Classify each as Corrective, Adaptive, or Perfective maintenance:
   (a) The application crashes when a school enters more than 50 classes.
   (b) The government introduces a new compulsory subject "Civic Education" that must appear on all timetables.
   (c) Users request a "Dark Mode" for the interface to reduce eye strain.
   (d) The database schema must be migrated from MySQL to PostgreSQL due to licensing policy changes.

   (3 marks)

EXERCISE 18 **Structured Question: Verification, Validation & Project Management – 15 marks**

A team is developing a Mobile Banking Application for a major Cameroonian bank.

1. Distinguish clearly between Verification and Validation. (2 marks)
2. For the Mobile Banking App, give one concrete example of a Verification activity and one concrete example of a Validation activity. (4 marks)
3. The project manager uses a PERT chart for scheduling. Explain the purpose of the Critical Path in PERT/CPM analysis. (2 marks)
4. During System Testing, a critical defect is found in the "Fund Transfer" module: transfers above 1,000,000 FCFA fail silently without notification. The release date is in 3 days.
   (a) Explain why this defect is classified as Critical severity. (2 marks)
   (b) Describe the Defect Life Cycle states this bug will go through from discovery to closure. (3 marks)

(c) Should the project manager release the software on time with a known critical defect? Justify your answer based on professional ethics and software quality standards. (2 marks)

10 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — MCQ: Life-Cycle Models**

Correct answer: C.
The **Waterfall model** (Royce, 1970) organises development as a strict linear sequence of phases: Requirements → Design → Implementation → Testing → Deployment → Maintenance. Each phase must be completed, documented and signed off before the next one starts, and going back to a previous phase is difficult and costly.
Why the other options are wrong:

- A describes models with overlapping phases and feedback loops (e.g. Spiral, Agile) — the opposite of Waterfall.
- B describes Agile/Scrum, with iterations called **sprints**.
- D describes the Prototyping model.

SOLUTION **Exercise 2 — True/False: Requirements Classification**

1. False. Issuing a book is a service the system must provide — a specific behaviour visible to the user. It is therefore a functional requirement.
2. False. A maximum response time of 2 seconds is a quality constraint on performance, not a function. It is a non-functional requirement (performance category).
3. True. A good **SRS** must be unambiguous (one single interpretation), complete (all requirements covered), consistent, verifiable (testable), traceable and modifiable. These are the standard IEEE quality criteria for an SRS.
4. False. Functional requirements describe what the system does (its services); it is the non-functional requirements that describe how well it performs them (speed, security, usability, reliability).

SOLUTION **Exercise 3 — Definitions: Key Concepts**

1. Software Verification: the set of static and dynamic activities that check that the software is being built correctly with respect to its specification at each stage — "Are we building the product right?" (e.g. reviews, inspections, unit tests against the design).

2. Software Validation: the activities that check that the final product meets the real needs of the customer/user — "Are we building the right product?" (e.g. acceptance testing with users).
3. Corrective Maintenance: maintenance performed to diagnose and fix faults (bugs) discovered after delivery, so that the software behaves as specified.
4. Perfective Maintenance: maintenance performed to improve or enhance the software after delivery — adding new features, improving performance or usability — even though no fault exists.
5. Critical Path: in PERT/CPM analysis, the longest path of dependent activities through the project network; it determines the minimum possible project duration. Any delay on a critical-path activity delays the whole project (zero slack/float).

SOLUTION **Exercise 4 — Direct Application: Testing Levels**

Level	Match	Justification
1. Unit Testing	C	Each module/function is tested alone, by the developer, often with s
2. Integration Testing	D	Modules are combined progressively and their interfaces and data e
3. System Testing	A	The complete, integrated system is tested against the SRS (function
4. Acceptance Testing	B	End-users/customer test the system in the real environment (alpha/

Memory aid: the **V-model** pairs each development stage with a testing level, from Unit (smallest) to Acceptance (closest to the user).

SOLUTION **Exercise 5 — Functional vs Non-Functional Requirements**

No.	Requirement (summary)	Class
1	Doctors can view a patient's medical history	F
2	Data encrypted at rest and in transit (AES-256)	NF
3	Monthly report of admitted patients per ward	F
4	Accessible via browsers on Windows, macOS, Linux	NF
5	Supports 500 concurrent users without degradation	NF
6	SMS alert to doctor on critical lab result	F
7	Automatic database backup every 24 hours	NF
8	Interface available in English and French	NF

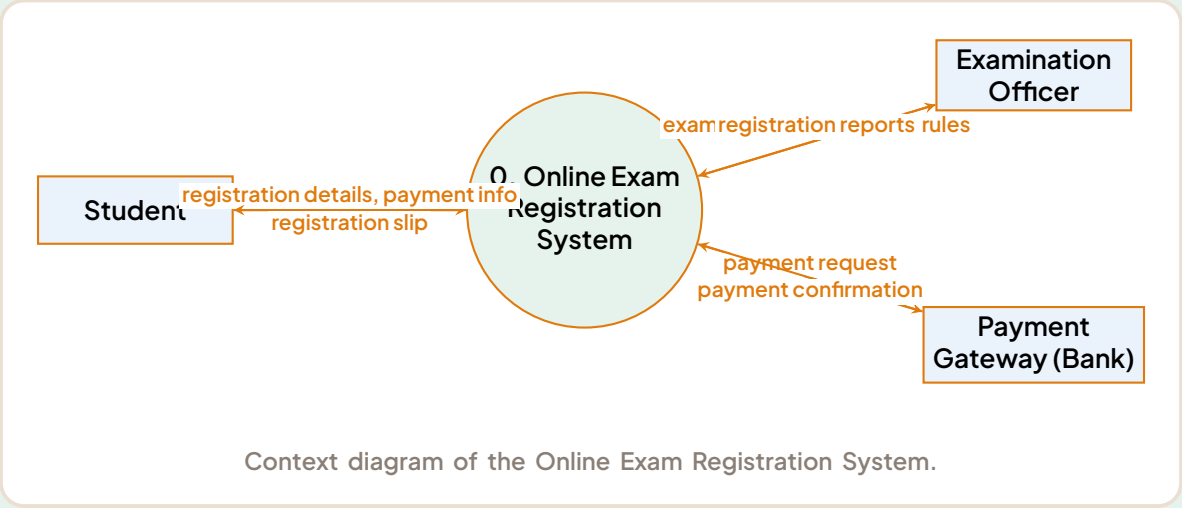
Justifications (two required):

- Requirement 1 (F): it describes a service/function the system must provide to a category of users — viewing the medical history is an observable behaviour, hence functional.
- Requirement 2 (NF): encryption imposes a security quality constraint on how data is handled; it does not add a user-visible service, hence non-functional (security category).

(Also acceptable: 5 is NF — performance/capacity; 4 is NF — portability; 7 is NF — reliability/operational constraint; 8 is NF — usability/localisation; 3 and 6 are F — concrete system services.)

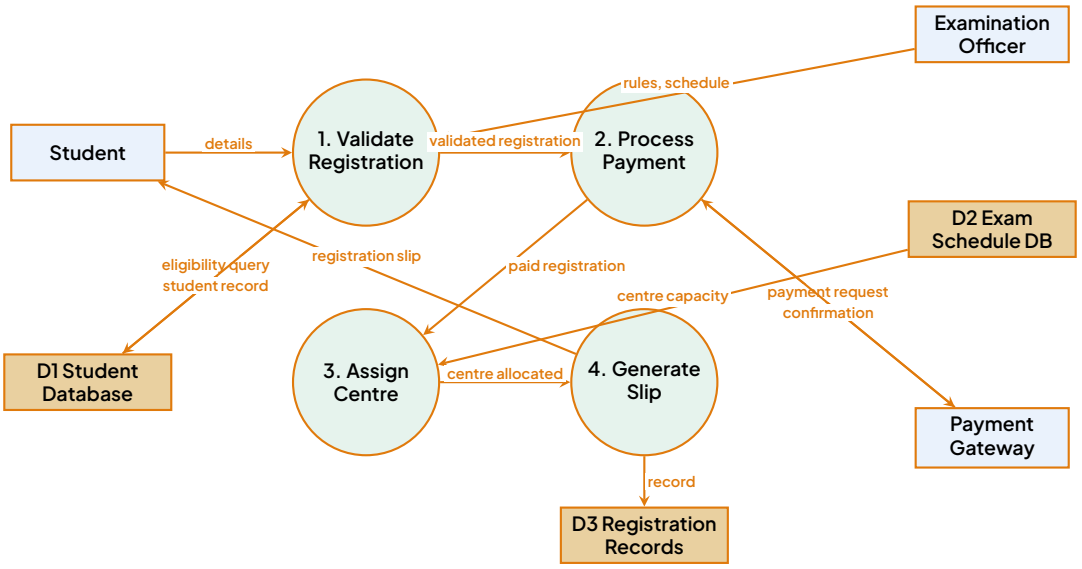
SOLUTION **Exercise 6 — Data Flow Diagram: Context & Level 1**

Context Diagram (Level 0): the whole system is one process; only external entities and their flows appear.



Context diagram of the Online Exam Registration System.

Level 1 DFD: process 0 is decomposed into the four processes, with the three data stores.



Level 1 DFD (Yourdon/Gane–Sarson style): circles = processes, open rectangles = data stores.

Marking points (typical): 3 external entities correctly placed (1 mk); 4 numbered processes (2 mks); 3 labelled data stores (1 mk); balanced, labelled flows with no direct entity-to-entity or store-to-store flow (2 mks).

SOLUTION **Exercise 7 — Test Plan Design**

We apply boundary value analysis (test the edges of each grade band: 0, 39, 40, 49, 50, 59, 60, 69, 70, 100) plus mid-range and error cases.

Test ID	Input	Expected	Type
TC01	85	A	Normal (mid-range of A)
TC02	70	A	Boundary (lower limit of A)
TC03	69	B	Boundary (just below A / top of B)
TC04	60	B	Boundary (lower limit of B)
TC05	50	C	Boundary (lower limit of C)
TC06	40	D	Boundary (lower limit of D)
TC07	39	F	Boundary (top of F)
TC08	0	F	Boundary (absolute minimum)
TC09	100	A	Boundary (absolute maximum)
TC10	-1	Error	Error (invalid, just below range)
TC11	101	Error	Error (invalid, just above range)
TC12	55	C	Normal (mid-range of C)

Comment: the most defect-prone points are the boundaries (a classic bug is writing `average > 70` instead of `average >= 70`, which would fail TC02). Invalid inputs just outside the range (-1, 101) are the strongest error tests. Any six well-chosen cases covering the three categories earn full credit.

SOLUTION **Exercise 8 — Gantt Chart Interpretation**

1. Total duration: the tasks are strictly sequential with no overlap, so the minimum project duration is the sum of the task durations: 2 + 3 + 5 + 2 = 12 weeks. (If the Gantt chart axis shows the Testing bar ending at week 14, this indicates a 2-week buffer or non-working period; the planned effort remains 12 weeks.)
2. Critical path: Requirements → Design → Coding → Testing. Since every task depends on the previous one, all tasks are critical (zero slack everywhere).
3. Design overruns by 2 weeks: Design is on the critical path, so Coding, Testing and Delivery all slip by 2 weeks: the end date moves from week 12 to week 14. There is no float to absorb the delay.
4. Longest phase: Coding (5 weeks).

SOLUTION **Exercise 9 — Essay: Development Approach Selection (15 marks)**

(a) Comparison (8 marks — 2 per criterion):

Criterion	Waterfall	Agile (Scrum)
Requirements volatility	Requirements must be fully specified and frozen up front; late changes are costly and disruptive.	Welcomes change; the product backlog is re-prioritised every sprint, ideal for unclear/evolving requirements.
Delivery timeline	One single delivery at the end (after all phases); nothing usable before month 3+.	Working increments delivered every 1–4 weeks; an MVP can be released within the first sprints.
Customer involvement	Customer seen mainly at the start (requirements) and end (acceptance).	Customer/Product Owner involved continuously; feedback after each sprint review steers the product.
Risk management	Risks surface late (testing at the end); high risk of building the wrong product.	Risks addressed early and continuously; each sprint is a small, inspectable experiment.

(b) Recommendation: Agile (Scrum) — 4 marks. Justification (any three):

1. Requirements are not fully known: Scrum's evolving backlog handles uncertainty, whereas Waterfall would freeze wrong assumptions.
2. The 3-month MVP deadline: time-boxed sprints guarantee a working, demonstrable product early enough to secure funding.
3. User feedback drives features: sprint reviews put real users (farmers, buyers) in the loop every iteration, matching the rapidly changing market.

(c) Two key practices (3 marks):

- Time-boxed sprints with a prioritised product backlog: the Product Owner orders features by business value, so the MVP's essential features are built first and the deadline is protected.
- Daily stand-up meetings + sprint review/retrospective: daily 15-minute meetings remove blockers immediately; the sprint review validates each increment with stakeholders, ensuring the MVP stays on track and on time.

Examiner expectation: a structured, balanced comparison (not a one-sided praise of Agile), a clear recommendation explicitly tied to the scenario facts, and concrete named practices.

SOLUTION **Exercise 10 — Case Study: Acquisition Strategy & Maintenance (15 marks)**

(a) Evaluation (9 marks — 1 per option per criterion, approx.):

Criterion	A: In-house	B: COTS	C: Open Source + contractor
Total cost of ownership	High: salaries, long development, testing, maintenance all borne by the Ministry.	High licence fees (often in foreign currency) + annual support; low initial development cost.	Low/moderate: no licence fee; pay contractor for adaptation and support.
Time to deployment	Slowest: full life cycle from scratch (many months/years).	Fastest: install, configure, train.	Intermediate: core engine exists; only adaptation needed.
Customisation (bilingual, shift schools)	Perfect fit: built exactly to Cameroonian rules.	Poor: foreign package may not handle bilingualism or shift schools; customisation limited/expensive.	Good: source code is open, so local rules can be implemented freely.
Vendor lock-in / sustainability	No lock-in; but depends on retaining skilled staff; risk of under-documentation.	Strong lock-in: vendor controls updates, prices, and may withdraw support.	Low lock-in: community + any local firm can maintain the code; sustainable.

(b) Recommendation (3 marks): Option C. It combines a fast deployment (proven engine such as FET), full customisability to the bilingual and shift-school specificities thanks to open source code, low cost in local currency, and no foreign vendor lock-in — a local contractor builds national capacity. (A well-argued Option A answer can also earn credit

if risks are acknowledged.)
(c) Maintenance classification (3 marks):

1. Crash beyond 50 classes → Corrective (fixing a defect).
2. New compulsory subject "Civic Education" → Adaptive (adapting to a change in the external/regulatory environment).
3. "Dark Mode" request → Perfective (improvement of usability, no fault).
4. Migration MySQL → PostgreSQL → Adaptive (adapting to a new technical/licensing environment).

SOLUTION **Exercise 11 — Structured Question: V&V and Project Management (15 marks)**

(a) Verification vs Validation (2 marks): Verification checks that the product is being built right — each work product conforms to its specification ("Did we build the system right?"). Validation checks that the right product is being built — the final system satisfies the user's actual needs ("Did we build the right system?").
(b) Concrete examples (4 marks):

- Verification: a code review / unit test checking that the "Fund Transfer" module correctly implements the design rule "debit account A, credit account B, log the transaction" — i.e. conformance to the specification.
- Validation: user acceptance testing where real bank customers perform transfers, check balances and confirm SMS receipts in a pilot agency, confirming the app meets their real banking needs.

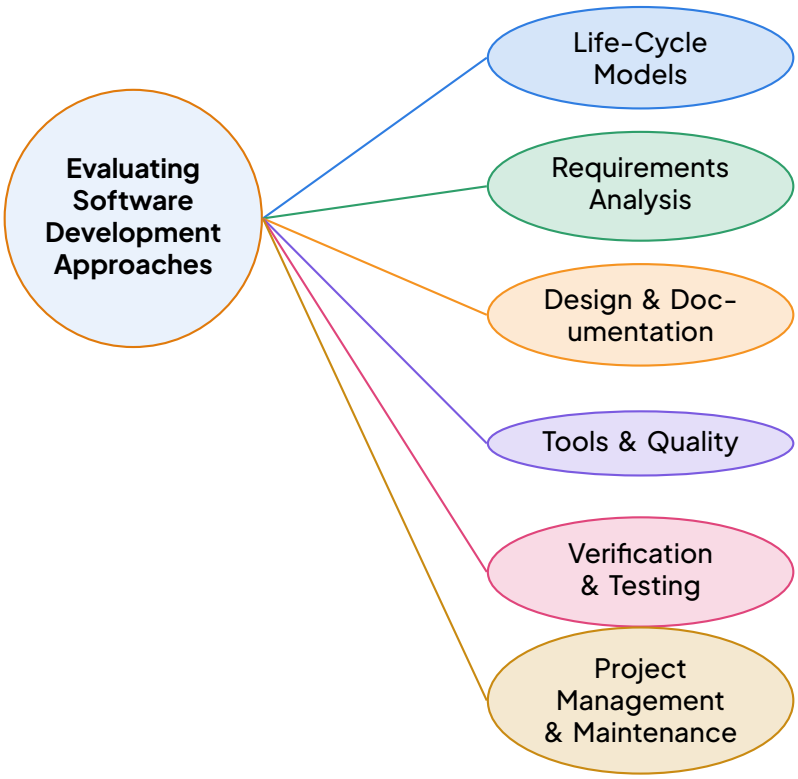
(c) Purpose of the Critical Path (2 marks): the critical path is the longest chain of dependent activities; it gives the minimum project duration and identifies activities with zero slack. The manager monitors it closely because any delay on it delays delivery, and resources can be shifted from non-critical tasks (which have float) to protect the deadline.
(d) The critical defect:

1. Why Critical severity (2 marks): a core business function (large transfers) fails silently — money may be blocked or unaccounted for with no error message, causing financial loss and loss of customer trust; there is no workaround. Severity is about impact, and here the impact on data integrity and money is maximal.
2. Defect life cycle (3 marks): New/Open (tester reports the bug with steps to reproduce) → Assigned (to a developer) → In Progress/Fixed (developer corrects the code) → Retest/Resolved (tester verifies the fix, including regression tests on transfers) → Closed (fix confirmed; otherwise it is Reopened).
3. Release decision (2 marks): No. Releasing banking software with a known criti-

cal defect violates professional ethics (duty to protect users' money and data) and software quality standards (a critical defect blocks release in any exit criteria). The responsible decision is to delay the release, fix and retest the module, or at minimum disable the faulty transfer feature — and transparently inform management and stakeholders.

Examiner expectation: precise V&V distinction with the classic "right" formulation, realistic banking examples, the full ordered defect-life-cycle chain, and an ethically argued refusal to ship with a known critical bug.

11 SUMMARY SHEET



Mind map of Chapter CSC07: Evaluating Software Development Approaches

FICHE BILAN **L'essentiel à connaître pour l'examen**

Key Definitions

- **Software Development Life Cycle (SDLC)**: Structured sequence of phases (planning, analysis, design, implementation, testing, deployment, maintenance) guiding software production.
- **Requirements Specification (SRS)**: Formal document describing functional and non-functional requirements, constraints, and acceptance criteria.
- **Software Design**: Transformation of requirements into architectural (high-level) and detailed (low-level) models using notations such as UML, DFD, ERD.
- **Verification**: Process ensuring the product is built right (conforms to specification) — static reviews, inspections, walkthroughs.
- **Validation**: Process ensuring the product is the right product (meets user needs) — dynamic testing, user acceptance testing.

- **Software Quality Attributes**: ISO 25010 characteristics — functionality, reliability, usability, efficiency, maintainability, portability.
- **Software Reuse**: Systematic use of existing assets (components, frameworks, patterns) to improve productivity and quality.
- **Project Management**: Planning, monitoring, controlling resources, schedule, risk, and communication to deliver on time and within budget.

Laws, Principles & Theorems (Examinable)

- Boehm's Cost-of-Change Curve: Cost to fix a defect rises exponentially the later it is detected (requirements → maintenance).
- Brooks' Law: Adding manpower to a late software project makes it later (communication overhead).
- Pareto Principle (80/20 rule): 80% of defects often originate from 20% of modules — guides testing focus.
- V-Model: Each development phase has a corresponding verification/validation phase (requirements ↔ acceptance test, design ↔ system test, etc.).
- Cyclomatic Complexity $M = E - N + 2P$: Quantitative measure of code complexity; guides test case design (basis path testing).
- ISO/IEC 12207: International standard for software life-cycle processes — often referenced for process evaluation.

Methods (Step-by-Step)

METHOD **Requirements Elicitation & Specification**

1. Identify stakeholders and conduct interviews, workshops, observation.
2. Document functional requirements (use cases, user stories) and non-functional requirements (performance, security).
3. Prioritise using MoSCoW (Must, Should, Could, Won't).
4. Write SRS following IEEE 830 structure; review with stakeholders; baseline.

METHOD **Software Design (Two-Stage)**

1. Architectural Design: Define subsystems, interfaces, data flow, deployment (UML component, deployment diagrams). Apply patterns (MVC, layered, microservices).
2. Detailed Design: Specify algorithms, data structures, class diagrams, sequence diagrams, database schema (ERD). Produce design specification document.

METHOD **Verification & Validation Strategy**

1. Static Verification: Reviews, inspections, static analysis tools (lint, SonarQube).
2. Unit Testing: White-box (basis path, condition coverage) using frameworks (JUnit, pytest).
3. Integration Testing: Incremental (top-down, bottom-up, sandwich) with stubs/drivers.
4. System Testing: Black-box (equivalence partitioning, boundary value, state transition).
5. Acceptance Testing: Alpha/beta, user acceptance test (UAT) against SRS.

METHOD **Project Monitoring (Earned Value)**

1. Define Work Breakdown Structure (WBS) and assign Planned Value (PV).
2. Track Earned Value (EV) and Actual Cost (AC) each reporting period.
3. Compute CPI = EV/AC, SPI = EV/PV; forecast Estimate at Completion (EAC).

Common Mistakes (Traps)

WARNING **Frequent Errors**

- Confusing verification with validation — remember: verification = "built right", validation = "right product".
- Ignoring non-functional requirements in SRS; they become costly to add later.
- Over-designing early (analysis paralysis) or under-designing (code-and-fix).
- Writing test cases after code instead of deriving from specifications (test-last vs test-first).
- Assuming adding developers speeds up a delayed project (Brooks' Law).
- Neglecting configuration management and version control in team settings.
- Forgetting to update traceability matrix when requirements change.

GCE Examination Tips

- Define terms precisely (e.g., "software quality" = conformance to requirements + fitness for use).
- Compare life-cycle models (Waterfall, V-Model, Incremental, Spiral, Agile) in a table: phases, risk handling, flexibility, suitability.
- Draw a V-Model diagram linking each development phase to its test phase — often asked for 4–6 marks.
- Calculate cyclomatic complexity from a control-flow graph; design basis path test cases.
- Interpret earned-value metrics (CPI, SPI) and state corrective actions.
- Justify choice of a model for a given scenario (e.g., safety-critical → V-Model; rapidly changing requirements → Agile).
- Use Cameroonian context where natural: e.g., a mobile-money app for MTN/Orange requiring high reliability and regulatory compliance.
- Time management: Allocate 10 min for a 10-mark essay, 5 min for a 5-mark diagram, 2 min per short definition.

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+